DICE: Decentralized Incentive-driven Federated Learning for Efficient & Scalable Customization of Pretrained Models in Web3

Hong Lin
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
honglin@zju.edu.cn

Zhongle Xie
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
xiezl@zju.edu.cn

Lidan Shou*
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
should@zju.edu.cn

Ke Chen
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
chenk@zju.edu.cn

Gang Chen
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
cg@zju.edu.cn

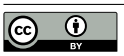
Abstract

Customizing Pretrained Models (PMs) for downstream tasks demands task-specific data, which burgeoning Web3 applications provide abundantly. Blockchain-powered federated learning (BCFL) offers a secure and privacy-preserving paradigm to customize PMs among *self-interested* and *untrusted* Web3 participants. Yet, existing BCFL algorithms face severe *efficiency* and *scalability* challenges when applied to large PMs, due to their substantial computational and memory requirements and the *hardware constraints* of many participants. This paper introduces DICE, a novel BCFL algorithm that addresses the efficiency and scalability challenges of customizing large PMs in Web3 via two sets of synergistic techniques: (i) a *dynamic progressive learning* scheme and a *model slice verification* scheme to enable a wider range of honest participants, including those unable to deploy the full model, to engage in customizing PMs, while preserving model utility; (ii) a *tree-structured committee consensus* protocol and an *efficiency-driven incentive* scheme to address the dominant bottleneck that hinders existing BCFL algorithms from scaling to a larger number of data holders for customizing data-hungry large PMs, while maintaining security. Theoretical analysis and extensive experimental results on customizing vision and language PMs demonstrate DICE's efficiency, effectiveness, and security. A comprehensive technical report of DICE including its implementation details, extended analyses, discussions, and experiments is available at https://github.com/Hoho-Lin/DICE.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

*Lidan Shou is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '26, Rome, Italy
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2539-5/2026/11
https://doi.org/10.1145/3799682.3840574

Keywords

Federated Learning, Pretrained Models, Blockchain, Web3

ACM Reference Format:
Hong Lin, Zhongle Xie, Lidan Shou, Ke Chen, and Gang Chen. 2026. DICE: Decentralized Incentive-driven Federated Learning for Efficient & Scalable Customization of Pretrained Models in Web3. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26), November 07–11, 2026, Rome, Italy*. ACM, New York, NY, USA, 23 pages. https://doi.org/10.1145/3799682.3840574

1 Introduction

Pretrained AI Models (PMs) and Web3 are converging to create more intelligent, automated, and efficient decentralized systems. These PMs require task-specific **customization** through further pretraining or fine-tuning before serving downstream tasks [13, 90, 116]. Meanwhile, burgeoning Web3 applications generate vast amounts of decentralized data [9, 29, 63, 93, 133], which offer immense potential to customize PMs. For instance, platforms such as MedSynapse [68] and Mind Network [74] produce terabytes of patient health data spread across millions of peer-to-peer devices. These data are valuable for customizing pretrained BERT [25] for disease diagnosis and treatment. However, aggregating private data on a single platform, whether centralized or decentralized, may violate increasingly stringent data protection regulations [102]. Moreover, Web3 participants are *self-interested* and *untrusted*: they are reluctant to contribute resources without monetary incentives and may disrupt customization tasks if malicious behavior is profitable [51, 57, 121].

Blockchain-powered Federated Learning (BCFL) [50, 54, 64, 75, 77, 82, 132] has emerged as a promising paradigm to support secure and privacy-preserving customization of a global model shared among Web3 participants. As shown in Fig. 1, a typical BCFL-based customization pipeline involves three participant roles: (i) a *model requester*, e.g., a medical institution, who needs a customized AI model and pays rewards to engage the other participants in customization; (ii) *workers*, e.g., MedSynapse's health data holders, who

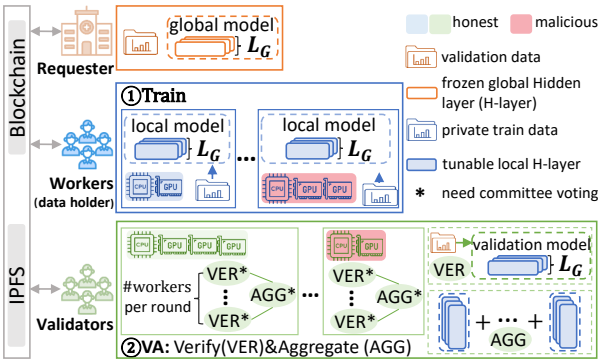


Figure 1: A general BCFL paradigm for customizing BERT-Large (with $L_G = 24$ H-layers, 340 million params), required by a medical institution for disease diagnosis, using health data in MedSynapse.

Table 1: Per-round memory costs of a worker (MC_w) and a validator (MC_v) in Fig. 1 under typical algorithms [56, 130].

Batch Size	8	16	32	64	**# Workers**	10	40	70	100
MC_w **(GB)**	11	17	30	59	MC_v **(GB)**	18	71	125	170

- These statistics are measured on NVIDIA A100 (PyTorch 2.6, CUDA 12.8) using the 2018n2c2 NER dataset [48], FP32 precision, Adam optimizer, sequence length 512, and batch size 8.
- MC_v is measured based on parallel verification of all updates without memory swapping or other techniques that increase verification latency [127].

train models using private data and submit *updates* (model parameters changed in training); (iii) *validators*, who verify and aggregate those updates. BCFL algorithms generally coordinate participants via a blockchain network and proceed in *iterative rounds of training and verification & aggregation (VA) steps*. This paradigm relies on two critical components: (i) *a malicious participant handling scheme* [56, 130] that requires validators to verify worker updates and execute a consensus protocol to ensure security, and (ii) *an incentive scheme* [125] that encourages self-interested entities to participate and behave honestly, typically via monetary rewards, e.g., cryptocurrency [42].

However, existing BCFL algorithms are primarily designed for small models ($\leq$ tens of millions of parameters) [14, 56, 66], facing critical challenges in efficiency and scalability when applied to large PMs (typically $\geq$ 100 million parameters):

C1. Per-participant computational and memory overhead. Existing BCFL algorithms require every worker to perform training on the full model, and require every validator to perform multi-update verification [14, 54, 56, 66, 126]. As shown in Table 1, even for BERT-Large, an early-stage large pretrained model, these requirements already exceed the memory capacity of many commodity GPUs. Consequently, a large fraction of honest devices are excluded from effective participation.

C2. System-level scalability bottleneck. Since each validator in existing BCFL systems is required to verify all worker updates in every round [50, 77, 82], the per-validator costs and cross-validator consensus latency of those systems grow linearly with the worker scale (see Table 1). Consequently, the per-validator workload becomes a dominant bottleneck preventing those systems from scaling to more workers for customizing data-hungry large PMs.

C3. Limitations of existing efficiency techniques. Prior methods for efficient PM customization, such as LoRA [40], BitFit [11], and other parameter-efficient tuning (PEFT) techniques [131], can reduce worker-side costs but fail to address validator-side overhead. While model compression techniques such as distillation [37], pruning [55], and quantization [35] can reduce validator-side costs, they might sacrifice model quality [107, 127]. Critically, none of these techniques overcomes the above scalability bottleneck.

To address these challenges, this paper proposes DICE, a novel Decentralized InCEntive-driven BCFL algorithm for efficient, scalable, and secure customization of PMs in Web3. DICE supports lightweight training and verification of a large PM by two techniques built upon insights from existing efficient customization solutions: (i) a *Dynamic Progressive Learning (DPL)* scheme to reduce worker-side hardware resource requirements while maximizing the adaptability of PMs; and (ii) a *Model Slice Verification (MSV)* scheme to enable honest yet hardware-constrained participants to engage as validators, increasing the pool of potential validators and thus making malicious behavior less likely to succeed. More importantly, DICE breaks the linear dependency between per-validator workload and worker scale by two core innovations: (i) a *Tree-structured Committee Consensus (TCC)* protocol that decomposes the monolithic VA task into tree-structured sub-tasks and employs selective voting to reduce the mandatory workload required for security to only root and controversial sub-tasks; and (ii) an *Efficiency-Driven Incentive (EDI)* scheme that complements TCC by penalizing conflict generators, ensuring rational validators do not undermine TCC's efficiency gains. Jointly, TCC and EDI resolve the scalability bottleneck that prior BCFL systems cannot address even when integrated with efficient model customization techniques, while preserving robust security guarantees.

The contributions of this paper are outlined as follows:

• **Systematic Problem Analysis and Novel Algorithm.** We systematically analyze the efficiency and scalability challenges of customizing large PMs within existing BCFL systems, and propose DICE, a novel algorithm that significantly lowers the entry barriers of customization tasks, enabling a wider range of honest participants to contribute effectively. (§3)

• **Novel Technical Designs.** We introduce two sets of novel techniques in DICE: DPL&MSV to support lightweight training and verification of large PMs while preserving model utility; TCC&EDI to address the dominant bottleneck that hinders prior BCFL systems from scaling to a larger number of workers without compromising security. (§3.2–§3.3)

• **Theoretical Guarantees.** We provide a rigorous theoretical analysis demonstrating that DICE lowers the participation barrier for both workers and validators, achieves per-validator computational complexity that grows *sub-linearly* with the worker scale, and ensures that malicious behaviors are prevented with overwhelming probability. (§4)

• **Experimental Validation.** We conduct extensive experiments on customizing vision and language PMs to show that DICE significantly reduces both worker-side and validator-side costs compared to baselines, delivers competitive final model performance, and

maintains robust resistance against malicious actors. Additional experiments demonstrate that DICE is compatible with existing efficient customization techniques, accommodating diverse device constraints and model performance requirements without compromising security. (§5)

2 Preliminaries

We begin by presenting the system model, threat model, and foundational techniques from prior BCFL work on which DICE builds.

2.1 System Model

2.1.1 Participants. Similar to existing BCFL designs [56, 126], DICE involves three types of participants: model *requesters*, *workers*, and *validators*, as introduced in §1. Requesters need abundant data held privately by workers to customize PMs [15] for superior performance on downstream tasks, and possess powerful hardware to perform computation on large PMs. However, workers and validators may lack the hardware capacity to deploy a full model.

2.1.2 Coordination and Communication. DICE adopts a peer-to-peer *blockchain network* to coordinate participants. To initialize a customization task, the requester creates a smart contract that records task specifications (e.g., global model, validation dataset, algorithm hyperparameters) and manages cryptocurrency to reward honest participants who strictly follow the algorithm's logic [98, 104, 126, 130]. Next, workers and validators join the task by registering their unique identifiers, usually their network addresses, on this contract. During task runtime, all participants communicate with the smart contract to exchange data. For cost-efficient data exchange, we follow previous studies [19, 56, 126, 128] to integrate a *secure messaging scheme*. This scheme stores large-scale data that are too expensive to store on the blockchain (e.g., model parameters) in an additional storage system such as IPFS, persists data hashes and file handles in smart contracts, and ensures data availability and integrity by excluding participants from reward distribution if their file handles are unreachable or if their hashes, recomputed upon file retrieval, do not match the on-chain records. All messages are transmitted over authenticated channels using digital signatures, and messages sent repeatedly by honest participants are eventually delivered, though delays may be unbounded.

2.2 Threat Model

Threats to BCFL primarily come from two aspects: the coordination and communication infrastructures and algorithm participants [33].

2.2.1 Trust Assumptions. Following existing work [5, 6, 56, 65, 119, 126], we treat the blockchain network and IPFS as reliable underlying infrastructures and consider algorithm participants as *untrusted*, *self-interested*, yet *rational* players. Rational requesters have no incentive to deviate from the algorithm, as that would undermine their goal of customizing high-quality models and engaging other participants. We thus focus on threats from workers and validators.

2.2.2 Adversary Capacities. A malicious worker or validator may deviate from the algorithm in three forms: (i) *Poisoning*: submitting low-quality updates or corrupted VA results to degrade model performance [20, 24, 44, 56, 69, 86]; (ii) *Fraud* (free-riding): submitting a copy of honest participants' updates or VA results, which has only a negligible impact on model performance, to claim undeserved rewards [21–23, 51, 56, 121, 124]; (iii) *No response*: failing to submit results due to network issues, crashes, etc., which may stall the algorithm [56]. Malicious participants can launch attacks independently or collude, and may further amplify their impact via blockchain-layer malicious behaviors such as Sybil attacks [26] (controlling multiple fake identities) or DDoS attacks [33] (disrupting message delivery). Privacy inference attacks and their mitigation (e.g., via differential privacy) [33] are orthogonal to the algorithm-deviation threats we consider and outside the scope of this study.

2.3 Foundational Techniques Against Threats

DICE adopts a malicious participant handling scheme architecturally similar to existing BCFL systems [54, 56, 126, 130]. This scheme relies on validators to execute a *worker verification* step to detect active malicious workers and a *cross-validator consensus* protocol to prevent active malicious validators. As long as the consensus results are produced by honest validators, the system is secure against threats from active workers and validators. To handle non-responsive participants, this scheme integrates a *timeout* mechanism. Below, we review these base techniques and illustrate how DICE adopts or innovates beyond them.

2.3.1 Handling Malicious Workers. Worker verification aims to detect and reject invalid updates from malicious workers. Existing approaches fall into two categories: scoring-based (e.g., empirical thresholds on validation performance) and cryptography-based (e.g., zero-knowledge proofs) [33, 77]. DICE's MSV scheme builds upon the former category (since the latter cannot directly assess model quality) while remaining compatible with the latter. Unlike existing scoring-based schemes that require full model verification, MSV is specifically designed to enable validators who cannot deploy full models to engage. This increases the total resources that can be leveraged for malicious behavior prevention and raises the difficulty of successful attacks.

2.3.2 Handling Malicious Validator. Consensus protocols are the core component for defending against malicious validators. These protocols generally fall into three paradigms [132]: proof-based [71, 88], committee-based [4, 53], and hybrid. DICE's TCC protocol builds upon the hybrid paradigm that employs *economic staking* as proof of eligibility to participate and *committee voting* to determine the valid results [56, 126]. This paradigm offers superior resource efficiency over proof-based alternatives, and unlike committee-based paradigms deployed in permissioned environments, enables open participation in customization tasks for anyone willing to contribute data or hardware resources. In contrast to existing protocols where each validator must process all worker updates, TCC and associated EDI are designed to break the linear dependency between per-validator workload and worker scale, allowing DICE to scale to more workers for better customization performance.

Standard consensus protocols may lack inherent mechanisms that verify the authorship of the received results to prevent validator fraud attacks. To close this gap, a common solution [56, 62, 101] is to integrate a *cryptographic commitment scheme* into the consensus process. This scheme requires validators to submit local computation results in two phases: *committing* and *revealing*. When

Table 2: Notation Summary.

Symbol	Definition
r, R, η	global round index; total number of rounds; learning rate
w, v	worker's and validator's addresses on the blockchain network ♠
$\mathbb{W}_{all}, \mathbb{V}_{all}$	the complete sets of workers and validators
$\mathbb{W}^r, \mathbb{V}^r$	the sets of active workers and validators in round r ♦
θ^r, θ^r_w	global model parameters and worker w's update in round r
$\mathbb{VAR}_v$	the set of results produced by validator v for a VA sub-task
L_G, L_E	H-layer counts in the global model and the emulator
L_w, L_v	deployable H-layer counts on worker w and validator v
$\mathbb{M}^r_w$	model slice selected to be trained on worker w in round r
$\mathbb{LUS}_j$	j-th subset of the slice $\cup_{w\in\mathbb{W}_{all}}\mathbb{M}^r_w$
N_{lus}	number of slices $\mathbb{LUS}_j$
$\theta^r_{\cdot,\mathbb{LUS}_j}$	$\mathbb{LUS}_j$'s parameters from any worker or set of workers
N_c, N_t	integer thresholds used in the TCC protocol
$N^{all}_{v,t}$	number of VA sub-tasks executed by validator v in a round
$N_{v,t}$	number of VA sub-tasks v correctly completed in a round
$N^{post}_{v,t}$	number of sub-tasks v correctly completed after conflicts arise
E_{total}	total amount of security deposit staked by all validators
E_c	expected amount of stake selected for committee formation
ϵ_j, ϵ_c	model performance threshold; voting threshold
WR, VR	rewards for workers and validators in each round
$\lvert\cdot\rvert, \cdot \Vert \cdot$	element count in a set; hash input concatenation
$\Omega(\cdot), O(\cdot)$	asymptotic lower bound; asymptotic upper bound
$\Theta(\cdot)$	asymptotically tight bound

♠ Each worker/validator is represented by its unique address.
♦ "Active" means having submitted an update or VA results in the current round.

committing, validators submit *fingerprint* and *commitment* generated by hashing raw results. Only after a consensus fingerprint is determined will validators reveal the complete raw results. DICE adopts this scheme with a slight modification to accelerate TCC.

2.3.3 Handling Non-Responsive Participants. DICE inherits a timeout scheme from prior work [56] to prevent deadlock caused by non-responsive participants. This scheme sets a timeout for each stalling-prone task state, dynamically adjusting it based on the moving average of each state's actual execution time, and assumes that at least one honest participant who has access to the task's smart contract remains active in each BCFL task. When any state exceeds its timeout, any active participant can invoke the smart contract to terminate the task and let the model requester decide whether to restart the task or just acquire the global model customized so far and reclaim the remaining funds. The scheme is fully compatible with DICE's novel designs and thus is directly incorporated.

3 The DICE Algorithm

This section presents our DICE algorithm. §3.1 first provides a high-level overview of DICE's workflow. Then, §3.2–§3.3 introduce two sets of techniques that DICE utilizes to improve efficiency and scalability while maintaining effectiveness and security of model customization in Web3. Table 2 summarizes the main notations used in this paper.

3.1 Overview

Fig. 2 presents an overview of DICE for customizing the PM, which comprises an embedding layer, a sequence of intermediate hidden layers (H-layers), and an output layer. DICE executes every customization task in iterative rounds. Each round contains a *training* step, where workers produce updates for a global model, and a *VA* step, where validators verify worker updates and aggregate qualified updates to produce a new global model. In the training step, DICE uses a DPL scheme to enable hardware-constrained workers, especially those having abundant task-specific data, to engage. This scheme divides the training step into two states: PREPARATION and TRAINING. In the VA step, DICE adopts MSV, TCC, and EDI to reduce validator-side costs while maintaining security. TCC, integrated with a cryptographic commitment scheme, structures the VA step into VA_COMMIT and VA_REVEAL states.

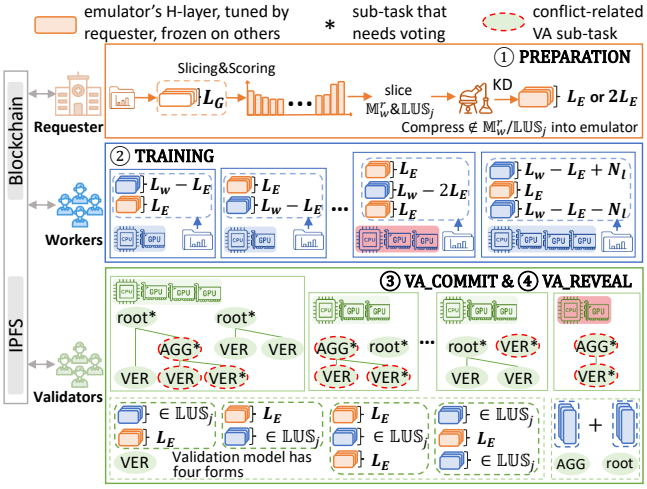


Figure 2: The overview of DICE.

Algorithms 1–2 present core procedures of DICE in four states. In PREPARATION, the start state of each round, the requester prepares necessary materials for the subsequent lightweight training and verification based on our DPL and MSV schemes (Alg. 1 lines 1–8, Alg. 2 lines 1–4). RequesterOnchainProcedure() changes the task state to TRAINING upon receiving those materials. Then, workers produce updates for the global model cost-effectively (Alg. 1 lines 9–13, Alg. 2 lines 5–9). After a certain number of workers have submitted updates, WorkerOnchainProcedure() creates a VA task forest based on our TCC scheme and changes the task state to VA_COMMIT. During VA_COMMIT, validators produce and submit fingerprints and commitments of VA results (Alg. 1 lines 14–22). ValidatorOnchainProcedure() checks whether validators reach consensus on the fingerprints of all VA sub-tasks (Alg. 2 lines 16–21), and if so, transitions the task state to VA_REVEAL. Then, validators reveal and verify the complete VA results (Alg. 1 lines 24–25, Alg. 2 lines 23–24). Finally, EDI() rewards honest participants and penalizes malicious validators (Alg. 2 line 25); then the customization task proceeds to the next round (line 26).

3.2 Lightweight Training and Verification powered by DPL and MSV

This section introduces lightweight training and verification procedures powered by our DPL and MSV schemes. The former aims to enable constrained workers to produce updates of a large PM and customize all necessary layers of the PM on worker data to maximize the PM's adaptation ability. The latter aims to enable a wider

Algorithm 1 DICE's off-chain procedures, i.e., computations executed locally by each participant.

1: **function** RequesterOffchainProcedure( ) ▷ prepare for §3.2
2: $state, r, r_g \leftarrow$ReadBlockchain()
3: **if** $state \neq$PREPARATION or $r \leq r_g$ **then** continue
4: $\{\mathbb{M}^r_w | w \in \mathbb{W}^r\}, \{\mathbb{LUS}_j | j \in [0, N_{lus})\} \leftarrow$ SliceSelection()
5: $\mathbb{M} \leftarrow \{\mathbb{M}^r_w | w \in \mathbb{W}^r\}; \mathbb{LUS} \leftarrow \{\mathbb{LUS}_j | j \in [0, N_{lus})\}$
6: $\{\theta^r_{\notin \mathbb{M}^r_w} | w \in \mathbb{W}^r\}, \{\theta^r_{\notin \mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ ModelDerivation()
7: $\mathbb{EM}_w, \mathbb{EM}_v, \mathbb{GM} \leftarrow$ RequesterIPFSStore()
8: RequesterOnchainProcedure($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
9: **function** WorkerOffchainProcedure( ) ▷ §3.2
10: $state, r, r_w \leftarrow$ReadBlockchain()
11: **if** $state \neq$TRAINING or $r \leq r_w$ **then** continue
12: $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ DPLtrain() ▷ layers $\notin \mathbb{M}^r_w$ are frozen
13: $\mathbb{LU} \leftarrow$ WorkerIPFSStore(); WorkerOnchainProcedure($\mathbb{LU}$)
14: **function** ValidatorOffchainProcedure( ) ▷ §3.2&§3.3
15: $state \leftarrow$ReadBlockchain()
16: **if** $state \notin$ {VA_COMMIT, VA_REVEAL} **then** return
17: $\mathbb{T}_{VA}, \mathbb{T}_v \leftarrow$ ReadBlockchain(); ▷ $\mathbb{T}_v$ stores sub-tasks done by v
18: **if** $state =$ VA_COMMIT **then**
19: **while** $state =$VA_COMMIT & $|\mathbb{T}_v| \leq |\mathbb{T}_{VA}|$ **do**
20: $\mathbb{RES}^{commit}_v \leftarrow$ MSVorAGG()
21: ValidatorOnchainProcedure($\mathbb{RES}^{commit}_v$)
22: $state, \mathbb{T}_v \leftarrow$ ReadBlockchain()
23: **else if** $|\mathbb{T}_v| > 0$ **then**
24: $\mathbb{RES}^{reveal}_v \leftarrow$ RevealResPacking()
25: ValidatorOnchainProcedure($\mathbb{RES}^{reveal}_v$)

Algorithm 2 DICE's on-chain procedures, programmed in the smart contract and executed on the blockchain network.

1: **function** RequesterOnchainProcedure($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
2: **if** $state \neq$ PREPARATION or $r_g \geq r$ **then** return
3: RecordRequesterRes($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
4: $r_g \leftarrow r$; $state \leftarrow$ TRAINING
5: **function** WorkerOnchainProcedure($\mathbb{LU}$)
6: **if** $state \neq$ TRAINING or $r_w \geq r$ **then** return
7: $r_w \leftarrow r$; $\mathbb{AW}^r \leftarrow$ RecordWorkerRes($\mathbb{LU}$);
8: **if** $\mathbb{AW}^r = \mathbb{W}^r$ **then** ▷ sufficient active workers
9: CreateVATaskTree(); $state \leftarrow$ VA_COMMIT ▷ §3.3
10: **function** ValidatorOnchainProcedure($\mathbb{RES}_v$)
11: **if** $state \notin$ {VA_COMMIT, VA_REVEAL} **then** return
12: **if** $state =$VA_COMMIT **then** OnchainCommit($\mathbb{RES}_v$)
13: **else** OnchainReveal($\mathbb{RES}_v$)
14: **function** OnchainCommit($\mathbb{RES}_v$) ▷ §3.3
15: $tid, \mathbb{VAR}^{commit}_v, FP_v, CC_v, \pi_v, hash_v \leftarrow \mathbb{RES}_v$
16: **if** not Record($tid, \mathbb{VAR}^{commit}_v, FP_v, CC_v, \pi_v, hash_v$) **then** return
17: **if** NeedCommitteeVoting(tid) **then**
18: Voting($tid, \mathbb{VAR}^{commit}_v, FP_v, CC_v, \pi_v, hash_v$)
19: **else** CheckConflict($tid, \mathbb{VAR}^{commit}_v, FP_v, CC_v, \pi_v, hash_v$)
20: **if** $|\{v | v \in \mathbb{V}_{all}, |\mathbb{T}_v \backslash \mathbb{CR}| \geq \lceil \frac{|\mathbb{T}_{VA}| - |\mathbb{CR}|}{|\mathbb{T}_{VA}|} N_t \rceil\}| < N_c$ **then** return
21: **if** $|RES^r| \geq \mathbb{T}_{VA}$ **then** $state \leftarrow$ VA_REVEAL
22: **function** OnchainReveal($\mathbb{RES}_v$) ▷ §3.3
23: $\{< tid, \mathbb{VAR}^{reveal}_v > | tid \in \mathbb{T}_v\} \leftarrow \mathbb{RES}_v$
24: QualificationCheck($\{< tid, \mathbb{VAR}^{reveal}_v > | tid \in \mathbb{T}_v\}$)
25: **if** all($\{|\mathbb{T}_v| = 0 | v \in \mathbb{V}_{all}\}$) **then** EDI()
26: OnchainPreparation(); $state \leftarrow$ PREPARATION

range of validators, including those more hardware-constrained than workers, to verify updates, increasing the total hardware resources that can be leveraged to augment security. These objectives are achieved through the *Slice Selection* and *Large-to-Compact Model Derivation* procedures executed by the model requester. §3.2.1–§3.2.3 first illustrate how the requester executes these two procedures. Then, §3.2.4 describes the lightweight training and verification process on workers and validators. Finally, §3.2.5 discusses the design rationale behind our proposed schemes.

3.2.1 Slice Selection. In this procedure, the model requester selects a small slice of the global model for training on each worker based on the slice's importance score, and executes further-slicing for validators when necessary (Alg. 1 line 4).

For each worker w, the requester divides the global model into a set of slices $\mathbb{M}_{w,i}$ defined in Eq. 1, based on this worker's hardware capacity. Each slice contains a small number of H-layers.

$$\mathbb{M}_{w,i} = \begin{cases} \{1, \cdots, L_w - L_E\} & \text{if } i = 0 \text{ (shallow)} \\ \{1 + L_E + i, \cdots, L_w - L_E + i\} & \text{if } i \in [1, N'_s) \text{ (middle)} \\ \{L_G - (L_w - L_E - 1), \cdots, L_G\} & \text{if } i = N'_s \text{ (top)} \\ \{1, \cdots, L_w - L_E - (i - N'_s), \\ \ L_G - (i - N'_s - 1), \cdots, L_G\} & \text{if } i \in (N'_s, N_s] \text{ (combined)} \end{cases} \quad (1)$$

Here, the numbers in $\mathbb{M}_{w,i}$ denote the indices of global H-layers. L_G is the total number of H-layers in the global model. L_E is the number of H-layers in an emulator. L_w is the number of H-layers deployable on worker w, $N'_s = L_G - L_w$, $N_s = L_G - L_E - 1$.

Then, the requester computes each slice's importance score denoted by $Score_{w,i}$. Our DPL scheme defines this score by Eq. 2 with two main metrics: (i) $s^r_{\mathbb{M}_{w,i}}$, the performance of the global model after a slice $\mathbb{M}_{w,i}$ is tuned, and (ii) $div^r_{\mathbb{M}_{w,i}} = 1 - cos_sim(\theta^r_{\mathbb{M}_{w,i}}, \hat{\theta}^r_{\mathbb{M}_{w,i}})$, the divergence of the slice parameters before and after tuning.

$$Score_{w,i} = \frac{s^r_{\mathbb{M}_{w,i}}}{\max(\{s^r_{\mathbb{M}_{w,i}} | i \in [0, N_s]\})} + \frac{div^r_{\mathbb{M}_{w,i}}}{\max(\{div^r_{\mathbb{M}_{w,i}} | i \in [0, N_s]\})} \quad (2)$$

This design prioritizes training slices that yield greater performance improvement and exhibit more substantial parameter updates. The model requester selects $\mathbb{M}_{w,i}$ with the highest score and combines it with the output layer to form the final slice $\mathbb{M}^r_w$ for training on worker w.

Finally, the model requester executes further-slicing based on our MSV scheme to enable more validators to participate in verifying worker updates. Specifically, when any worker's update cannot be deployed together with an emulator in a lightweight validation model containing L_{vm} H-layers, the requester separates the complete set of worker-updated slices $\cup_{w \in \mathbb{W}_{all}} \mathbb{M}^r_w$ into non-overlapping sub-slices denoted by $\{\mathbb{LUS}_j | j \in [0, N_{lus})\}$. Here, L_{vm} is the number of H-layers that can be deployed on the most hardware-constrained validator, and $N_{lus} = \lceil \frac{|\cup_{w \in \mathbb{W}_{all}} \mathbb{M}^r_w \backslash \{output\}|}{L_{vm} - 2L_E} \rceil$. If the above condition does not hold, $N_{lus} = 1$ and $\mathbb{LUS}_0 = \cup_{w \in \mathbb{W}_{all}} \mathbb{M}^r_w$.

For every slice $\mathbb{M}^r_w$ and $\mathbb{LUS}_j$, the model requester compresses the global H-layers excluded from that slice into an emulator of a compact model by ModelDerivation() (Alg. 1 line 6).

3.2.2 Large-to-Compact Model Derivation. In this procedure, the model requester derives compact models from the global model for

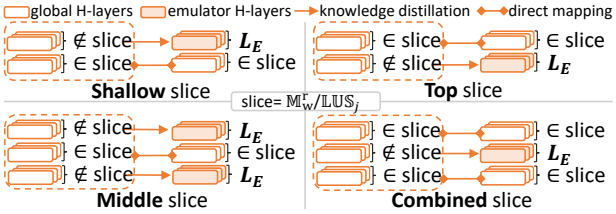


Figure 3: Emulator generation given slice $\mathbb{M}_w^r/\mathbb{LUS}_j$.

both workers and validators (Alg. 1 line 6). Each compact model retains the same embedding and output layers as the global model but contains a reduced number of H-layers. These compact models are designed to allow workers to efficiently generate updates for $\mathbb{M}_w^r$, and to enable validators to verify each update at a manageable cost. Motivated by offsite tuning [112], we insert a lightweight *emulator* into each compact model to ensure that each worker's update and the corresponding verification results closely approximate those obtained by training or verifying with the full model, thereby achieving performance comparable to customizing the full model. As presented in Fig. 3, the emulator is composed of L_E or $2L_E$ H-layers, with $L_E = 1$ by default and $2L_E$ H-layers adopted only when $\mathbb{M}_w^r$ or $\mathbb{LUS}_j$ is a middle slice. The emulator for worker w (or validator v) is produced by compressing the global H-layers not in $\mathbb{M}_w^r$ (or $\mathbb{LUS}_j$) through knowledge distillation (KD) [37, 112].

3.2.3 Runtime Mechanics. We now describe how the model requester executes the above procedures with two runtime mechanisms incorporated in DPL and MSV to balance model performance and computation efficiency.

(i) *Periodic Slice Reselection and Emulator Regeneration.* Rather than executing slice selection and large-to-compact model derivation in every round, we design a more cost-effective periodic mechanism that avoids unnecessary slice reselection or emulator regeneration and decouples the update frequency of the selected slices and the emulators based on their respective roles.

Under this periodic mechanism, the requester re-computes slice importance scores and selects new slices only when the average performance of the global model over the latest $\epsilon_r \in \mathbb{Z}^+$ rounds does not show distinct improvement. Otherwise, the requester skips slice selection and proceeds directly to the model derivation step, reusing the same slices as in the previous round. The rationale is that slice selection determines which part of the model to train, and a performance plateau signals the exhausted utility of further training the current selected slice and the necessity of exploring a different slice.

Moreover, the requester updates the emulator for a given worker or validator only when the output discrepancy between the full global model and the compact model built with the existing emulator exceeds a predefined threshold $\epsilon_{em} \in \mathbb{R}^+$. The rationale is that the emulator's role is to approximate the input-output mappings of the global H-layers excluded from the compact model, and a large output discrepancy implies the ineffectiveness of the current emulator for such approximation and the necessity of emulator regeneration.

With the above designs, the model requester re-selects slices and regenerates emulators in response to performance stagnation or model drift, respectively, thereby balancing model adaptability with computational efficiency.

(ii) *Adaptive Mode Switching.* In addition to the above periodic mechanism, DPL further incorporates an adaptive mode switching mechanism to accommodate heterogeneous hardware capabilities among workers. Under this mechanism, the requester dynamically switches between two operational modes of DPL. One is *Personalized* (P-DPL), where slices are selected per worker to maximize the utilization of available hardware, as described in §3.2.1. The other is *Unified* (U-DPL), where the same slice is selected for all workers by using a unified parameter $L_{wm} = \min(\{L_w | w \in \mathbb{W}_{all}\})$ to replace every worker's specific parameter L_w in §3.2.1. U-DPL is the default mode, since it requires the slice selection to be executed only once for all workers in any round and thus is cost-effective for requesters. The requester activates the P-DPL mode only if there exist hardware-rich workers (with $L_w > L_{wm}$) whose updates consistently achieve a performance score higher than $\frac{L_w}{L_{wm}} \cdot s_{wm}$, where s_{wm} is the best performance score that hardware-constrained workers (i.e., with $L_w = L_{wm}$) have ever achieved in all past rounds. Once this condition ceases to hold, the requester switches DPL back to its default mode.

This adaptive mechanism is motivated by the observation that the benefit of personalization depends on the alignment between hardware capacity and local training quality. Specifically, assigning larger slices to train on hardware-rich workers that fail to produce well-performing updates could lead to overfitting on those workers, reducing the generality of the global H-layers that hardware-constrained workers do not update, thereby compromising the final global model performance. Thus, P-DPL is preferred only when capable workers demonstrate consistently superior local performance exceeding a dynamic threshold.

Together, the two runtime mechanisms enable DICE to adapt its requester-side operations to both the model's learning dynamics and the participants' resource heterogeneity, ensuring that computational resources are invested only when they yield tangible benefits to model performance.

3.2.4 Lightweight Training and Verification. After the requester-side procedures complete, the worker constructs a compact local model for training, and the validator constructs a compact validation model for verification, as shown in Fig. 2. Each worker w executes DPLtrain() to produce the update for $\mathbb{M}_w^r$ and divide it into $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\}$ (Alg. 1 line 12). Each validator v executes MSVorAGG() to verify and aggregate $\theta^r_{w,\mathbb{LUS}_j}$ (Alg. 1 line 20). An update $\theta^r_{w,\mathbb{LUS}_j}$ will be considered valid only if it satisfies two conditions: (i) it contains at least one layer whose norm diverges from the historical norms of updates for that layer to pass the fraud check; (ii) its corresponding validation model performs better than a dynamically adjusted threshold ϵ_j to pass the poisoning check.

3.2.5 Discussion. We finally discuss the design rationales of DPL and MSV. Both schemes are motivated by the resource asymmetry between the model requester and the other participants, particularly the workers and validators who cannot deploy full models. They leverage the relatively abundant hardware resources of the requester to select slices and generate emulators for compact model execution on constrained participants, enabling those participants

to effectively contribute to customizing large PMs. By integrating two runtime mechanisms, namely (i) periodic slice reselection and emulator regeneration and (ii) adaptive mode switching, they slow down the growth of requester-side costs as the number of training rounds or the participant scale increases.

Meanwhile, the two schemes are designed to mitigate the potential biases introduced by using compact models in the training and verification stages, respectively. DPL mitigates training bias by optimizing information gain under heterogeneous hardware constraints among workers. The slice importance score quantifies the expected utility of training a specific set of layers by jointly considering the potential performance improvement and the magnitude of parameter updates. By selecting the slice that maximizes this score, DPL provides a principled solution to determine which part of the PM should be customized in each round, rather than relying on heuristic designs. MSV mitigates verification bias by employing a threshold ϵ_j, dynamically adjusted by the model requester based on the full global model's performance trend and actual distribution of the quality of worker updates, etc., to compensate for systematic under- or over-estimation of worker update quality.

Furthermore, DPL and MSV jointly make poisoning attacks more difficult. To successfully corrupt the final full model (e.g., via a carefully-crafted update that passes the honest validator's check and retains its toxicity across rounds), an adversary needs to anticipate not only the honest workers' concurrent updates but also the requester's future decisions on slice selection, emulator updates, and validation threshold (ϵ_j) adjustments, all of which are difficult to reliably predict at the time of attack injection. The tunability of ϵ_j also enables MSV to accommodate diverse and evolving adversarial behaviors, as the adjustment logic for ϵ_j can be refined when new attack strategies emerge.

3.3 Balancing Efficiency, Scalability, and Security by TCC and EDI

This section enhances the lightweight verification introduced in the last section with our TCC protocol and EDI scheme, two integrated techniques to further reduce validator-side computational complexity and maximize the system's ability to scale to more workers while ensuring security. §3.3.1 first presents how TCC breaks the linear dependency between each validator's workload and worker scale while generating correct consensus. Then, §3.3.2 presents how EDI complements TCC to strengthen the balance between efficiency, scalability, and security. Finally, §3.3.3 discusses the synergistic relationship between TCC and EDI.

3.3.1 Consensus Generation. TCC generates consensus over the entire VA task by four critical steps: Task Decomposition, Result Committing, Selective Committee Voting, and Result Revealing.

Task Decomposition (before `VA_COMMIT` state). In this step, a monolithic VA task is decomposed into N_{VA} sub-tasks in a VA task forest, where $N_{VA} = (2|\mathbb{W}^r| - 1)N_{lus}$ and $\mathbb{W}^r$ is the set of active workers in round r (Alg. 2 line 9). As shown in Fig. 2, the forest comprises several task trees. The leaf node in each $Tree_j$ represents a VER sub-task verifying a worker's update of $\mathbb{LUS}_j$. The non-leaf node represents an AGG sub-task aggregating qualified updates from two child nodes. Executing each non-leaf sub-task has a strong dependency on the results of its two child sub-tasks. For example, an AGG sub-task requires the performance scores computed in its child sub-tasks to determine each update's weight in aggregation. It may also need the file handles of the partial aggregated updates to retrieve those updates for further aggregation. After the VA task forest is constructed on the blockchain, result committing starts.

Result Committing (during `VA_COMMIT` state). Each validator iteratively executes the VA sub-task randomly selected from the forest based on its local hardware condition and submits the fingerprint and commitment of this sub-task's results to the blockchain based on a cryptographic commitment scheme to prevent fraud (Alg. 1 lines 18–22). The set of results from a validator v for a VA sub-task, denoted by $\mathbb{VAR}_v$, is produced as follows.

- For any VER sub-task, the validator first verifies a worker's update $\theta^r_{w,\mathbb{LUS}_j}$ as illustrated in §3.2.4, producing this sub-task's intermediate results: the outputs from the model built upon $\theta^r_{w,\mathbb{LUS}_j}$ on validation data, denoted by $V(\theta^r_{w,\mathbb{LUS}_j}, \mathbb{D}_{val})$. If the update is qualified, the validator then computes this update's weight in the final aggregated update, denoted by $\alpha_{w,\mathbb{LUS}_j}$. We define $\alpha_{w,\mathbb{LUS}_j}$ by the performance of the update $\theta^r_{w,\mathbb{LUS}_j}$ on validation data, while remaining compatible with alternative aggregation weight definitions, e.g., weights based on worker data volume [56, 67] and contribution to aggregated updates [30, 96]. Finally, the validator packs $\alpha_{w,\mathbb{LUS}_j}$ with $V(\theta^r_{w,\mathbb{LUS}_j}, \mathbb{D}_{val})$ into the result set.
- For any AGG sub-task, the validator first aggregates the qualified updates received from its two child nodes via Eq. 3–5, where a weight $\frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau_{c1,c2}}$ is used to ensure that the final result at the root node of $Tree_j$ yields a weighted average of the individual updates $\theta^r_{w,\mathbb{LUS}_j}$ from all workers $w \in \mathbb{W}_j$ who submitted valid updates for the slice $\mathbb{LUS}_j$. In this process, the validator produces the sub-task's intermediate results: $H(\beta_{\mathbb{W}_c,\mathbb{LUS}_j})$, $H(\tau_{c1,c2})$, $H(\frac{\beta_{\mathbb{W}_{c1},\mathbb{LUS}_j}}{\tau_{c1,c2}}\theta^r_{\mathbb{W}_{c1},\mathbb{LUS}_j})$, and $H(\frac{\beta_{\mathbb{W}_{c2},\mathbb{LUS}_j}}{\tau_{c1,c2}}\theta^r_{\mathbb{W}_{c2},\mathbb{LUS}_j})$, where $H(\cdot)$ is a cryptographic hash function. Finally, the validator packs the hash of the aggregated update, denoted by $H(\theta^r_{\mathbb{W},\mathbb{LUS}_j})$, and the above intermediate results into the result set.

$$\beta_{\mathbb{W}_c,\mathbb{LUS}_j} = \begin{cases} \alpha_{w,\mathbb{LUS}_j}, \text{where } w \in \mathbb{W}_c & \text{if } |\mathbb{W}_c| = 1 \\ 1 & otherwise \end{cases} \tag{3}$$

$$\tau_{c1,c2} = \begin{cases} \frac{1}{\sum_{w\in\mathbb{W}_j}\alpha_{w,\mathbb{LUS}_j}} & \text{if } |\mathbb{W}_{c1} \cup \mathbb{W}_{c2}| = |\mathbb{W}_j| \\ 1 & otherwise \end{cases} \tag{4}$$

$$\theta^r_{\mathbb{W},\mathbb{LUS}_j} = \sum_{c\in\{c1,c2\}} \frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau_{c1,c2}} \theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}. \tag{5}$$

Here, $c1$ and $c2$ denote two child nodes. $\mathbb{W}_c$ is the set of workers whose updates are aggregated in any child node $c \in \{c1, c2\}$. $\theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}$ is the update from any child node c, being either a specific worker's update if $\mathbb{W}_c$ is a singleton or a partial aggregated update. $\mathbb{W} = \mathbb{W}_{c1} \cup \mathbb{W}_{c2}$.

The conventional commitment scheme [56] defines the fingerprint and commitment of each result set $\mathbb{VAR}_v$ by $FP_v = H(\mathbb{VAR}_v)$ and $CC_v = H(v||\mathbb{VAR}_v)$, respectively. TCC extends this scheme by including a subset of raw results $\mathbb{VAR}_v^{\text{commit}}$ ($\subset \mathbb{VAR}_v$) in the fingerprint. $\mathbb{VAR}_v^{\text{commit}}$ includes the weight $\alpha_{w,\mathbb{LUS}_j}$ for any VER

sub-task, or the hash of the aggregated update $H(\theta^r_{\mathbb{W},\mathbb{LUS}_j})$ for any AGG sub-task. TCC also requires validators that execute any AGG sub-task to submit the file handle of the aggregated update along with fingerprints and commitments. These designs enable validators to reuse child-node results without re-execution, thus reducing individual workload and global consensus latency across the VA task forest.

For any given sub-task, honest validators will submit the *same* valid fingerprint, while malicious ones may submit invalid versions. `ValidatorOnchainProcedure()` feeds each validator v's submission into `OnchainCommit()` for validity checking and consensus fingerprint generation (Alg. 2 lines 12, 16–21). For a non-leaf sub-task submission from validator v, this function first checks if the blockchain records v's self-produced results for all descendant sub-tasks of this non-leaf node. If v lacks self-produced results for any descendant, the procedure then checks whether the blockchain already contains consensus results for those specific descendants produced by other validators. If neither condition is met, v's submission is discarded (line 16). This ensures honest validators can produce valid results for non-leaf tasks without redundant re-execution of that sub-task's descendants. Submissions for all leaf sub-tasks, as well as those for non-leaf sub-tasks passing the above check, proceed to selective committee voting for validity determination.

Selective Committee Voting (during `VA_COMMIT` state). Fig. 4 depicts TCC's selective voting strategy (Alg. 2 lines 17–19). Committees are formed only for root sub-tasks at the start of the consensus process (Fig. 4①) to prevent malicious validators from prematurely satisfying the global consensus conditions by using quickly forged results without actual execution of any sub-task. Additional committees are formed dynamically when a sub-task becomes controversial (Fig. 4③). Committee formation requires each validator to stake at least $E_{\min}$ units of cryptocurrency as a security deposit upon joining a customization task. Each committee, selected randomly from all validators with probability proportional to their stakes, votes to determine the consensus fingerprint of its assigned sub-task.

`Voting()` is triggered only if the sub-task is root or controversial (Alg. 2 line 18). This function first verifies validator v's committee membership and calculates v's votes using its submitted proof π_v and hash value $hash_v$. For details of this verification process, please refer to prior work [56]. A validator v's votes are counted only if it meets two requirements: (i) For a root non-controversial sub-task, v must have committed to at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ *conflict-unrelated* sub-tasks, where N_{CF} is the number of *conflict-related* sub-tasks including controversial sub-tasks and their descendants, $N_t \in \mathbb{Z}^+$ is a predefined threshold. (ii) For a controversial sub-task, v must have committed to all descendant sub-tasks of that controversial task. The former requirement ensures that consensus on all conflict-unrelated tasks is produced by honest validators (formally proved in Theorem 4 of §4.2), while the latter one ensures that all controversial tasks receive correct inputs from their descendants. If v's votes are eligible, the function then casts them toward the candidate fingerprint that matches v's committed fingerprint. The consensus fingerprint for a root or controversial sub-task is the highest-voted fingerprint that receives more than $E_c \cdot \epsilon_c$ votes.

For each non-root and non-controversial sub-task where voting is unnecessary, `CheckConflict()` is invoked (Alg. 2 line 19). If no

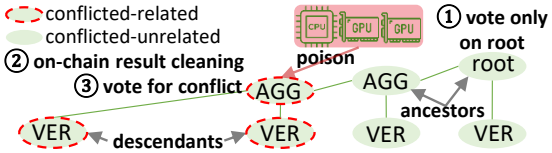


Figure 4: Selective committee voting.

conflict is detected, the received fingerprint is directly adopted as the sub-task's consensus. Otherwise, that sub-task is marked controversial. Then, that sub-task and its descendants become conflict-related, triggering a two-step resolution procedure. This procedure first removes all previous results of that sub-task and its ancestors from the blockchain (Fig. 4②), preventing that sub-task from compromising the validity of its ancestors' results. Then, a new committee is randomly selected from all validators to vote on that sub-task's consensus fingerprint (Fig. 4③).

Given predefined thresholds $N_t \in \mathbb{Z}^+$, $N_c \in \mathbb{Z}^+$, and the number of conflict-related sub-tasks N_{CF}, the global consensus is established when (i) at least N_c validators have fingerprints recorded on the blockchain, and if *conflict-unrelated* sub-tasks exist, each validator's records cover at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated sub-tasks, and (ii) every sub-task has determined consensus fingerprints (Alg. 2 lines 20–21). After that, the result revealing starts.

Result Revealing (during `VA_REVEAL` state). Each validator reveals unexposed raw results $\mathbb{VAR}_v^{\text{reveal}} = \mathbb{VAR}_v \setminus \mathbb{VAR}_v^{\text{commit}}$ (Alg. 1 lines 24–25). Then, `OnchainReveal()` checks whether the following conditions are all satisfied: (i) $H(v \parallel \mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) = CC_v$, (ii) $H(\mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) = FP_{\text{agreed}}$, (iii) $\mathbb{VAR}_v^{\text{commit}} = \mathbb{VAR}_{\text{agreed}}^{\text{commit}}$ (Alg. 2 lines 22–26). Here, FP_{agreed} and $\mathbb{VAR}_{\text{agreed}}^{\text{commit}}$ are the consensus results for the sub-task to which $\mathbb{VAR}_v^{\text{reveal}}$ pertains. We use the above check to prevent validator fraud. For any malicious validator v', any attempt to reuse a CC_v belonging to a different validator $v \neq v'$ will violate the first condition, as CC_v is bound to v's identity. Moreover, during `VA_COMMIT`, no validator can compute a commitment that binds to its own identity without the knowledge of the complete $\mathbb{VAR}_v$. Thus, only honest validators who actually computed $\mathbb{VAR}_v$ can satisfy all three conditions simultaneously. The formal security guarantee of our commitment scheme is provided in Theorem 3 of §4.2. Revealing ends when all validators that participated in committing have revealed their complete raw results (line 25).

3.3.2 Reward and Punish. At the end of `VA_REVEAL`, our EDI scheme allocates rewards and penalties to participants (Alg. 2 line 25), incentivizing honest behavior and encouraging greater hardware resource contributions to accelerate customization in `DICE`.

Rather than replacing existing incentive schemes [42, 125], our EDI scheme augments them with workload-sensitive coefficients that proportionally amplify cryptocurrency rewards for: (i) honest workers who contribute qualified updates across more model parameters, and (ii) honest validators who correctly complete more VA sub-tasks. Eq. 6 computes the worker w's workload-sensitive coefficient $Final_w$. Eq. 7 and Eq. 8 compute the validator v's workload-sensitive coefficients $Final_v^{pre}$ and $Final_v^{post}$, corresponding to the stages before and after conflicts arise, respectively. For any worker

or validator, its workload-sensitive coefficient(s) will be adjusted to zero if it is detected as a malicious participant. Eq. 9 and Eq. 10 compute the worker's and validator's corresponding payoff R_w and R_v. These designs ensure that participants are incentivized to improve system efficiency while preserving fairness and effectiveness guarantees of the underlying incentive scheme.

$$Final_w = \Phi_w \cdot (1 + \gamma_{worker} \frac{\sum_{j=0}^{N_{lus}-1} Contri_{w,j}}{\sum_{w' \in \mathbb{W}^r} \sum_{j=0}^{N_{lus}-1} Contri_{w',j}}), \quad (6)$$

$$Final_v^{pre} = \Phi_v \cdot (1 + \gamma_{validator} \frac{(N_{v,t} - N_{v,t}^{post})}{\sum_{v' \in \mathbb{V}^r} (N_{v',t} - N_{v',t}^{post})}), \quad (7)$$

$$Final_v^{post} = \Phi_v \cdot (1 + \gamma_{validator} \frac{N_{v,t}^{post}}{\sum_{v' \in \mathbb{V}^r} N_{v',t}^{post}}), \quad (8)$$

$$R_w = \frac{Final_w}{\sum_{w' \in \mathbb{W}^r} Final_{w'}} \cdot WR, \quad (9)$$

$$R_v = [\frac{Final_v^{pre}}{\sum_{v' \in \mathbb{V}^r} Final_{v'}^{pre}} \cdot (1 - \frac{N_{CF}}{N_{VA}}) + \frac{Final_v^{post}}{\sum_{v' \in \mathbb{V}^r} Final_{v'}^{post}} \cdot \frac{N_{CF}}{N_{VA}}] \cdot VR \quad (10)$$

Here, Φ_w, Φ_v are base rewards for worker w and validator v computed via the underlying incentive mechanism. $\gamma_{worker}, \gamma_{validator} \in (0, 1]$ are tunable parameters to adjust intensity of workload incentive. $Contri_{w,j}$ is worker w's contribution score to the j-th model slice, following the worker contribution definition of the underlying incentive scheme. $N_{v,t}$ is the total number of VA sub-tasks that validator v correctly completes in the current round, and $N_{v,t}^{post}$ ($\leq N_{v,t}$) is the number of those completed after conflicts arise. WR and VR are total rewards for workers and validators in a single round, respectively.

With the above design, validators may deliberately trigger conflicts for extra time to complete more sub-tasks and thus gain more rewards, which would increase consensus latency. To preempt this, EDI penalizes the validator who submits invalid results for a sub-task (i.e., conflicting with that sub-task's final consensus) by burning a portion of its staked deposit. Eq. 11 computes the amount $Punish$ burnt for each such validator, where $\epsilon_{penalty} \in \mathbb{Z}^+$ is a pre-defined base penalty.

$$Punish = \epsilon_{penalty} + \frac{N_{CF}}{N_{VA}} \cdot VR. \quad (11)$$

Since $Punish$ exceeds $\frac{N_{CF}}{N_{VA}} \cdot VR$ (the maximum extra reward obtainable by triggering a conflict) and scales linearly with the number of conflict-related sub-tasks, any deliberate conflict-triggering behavior yields strictly negative expected utility, regardless of whether validators collude and how many sub-tasks are controversial in a round. This design makes submissions of controversial results economically irrational and preemptively eliminates the latency in conflict resolution.

3.3.3 Discussion. We finally conclude this section by discussing how TCC and EDI achieve a balance between efficiency, scalability, and security. TCC's core insight is that global consensus does not require every validator to verify every update. By VA task decomposition and selective committee voting, TCC ensures that honest validators execute only a random subset of sub-tasks, while the protocol still produces a correct global consensus. This shifts the scalability bottleneck from individual capabilities to the collective power of the validator set, enabling the system to scale to more workers as the validator scale grows rather than being limited by individual constraints. However, a standalone TCC might degenerate if malicious validators trigger excessive conflicts. EDI prevents this by burning the stake of each malicious validator who submits invalid VA results, with penalties exceeding potential gains, thus ensuring rational validators abstain from provoking conflicts. With EDI disincentivizing conflicts, we will show in §4.2 that, by setting N_t, N_c, E_c, ϵ_c properly, TCC can minimize per-validator computational complexity in the consensus process while preventing malicious behaviors with high probability.

4 Theoretical Analysis

This section first analyzes DICE's efficiency in §4.1 and its security against malicious workers and validators in §4.2, then discusses a fundamental trade-off revealed from these analyses in §4.3. In §4.1–§4.3, we assume that all participants are active. §4.4 discusses the impacts of non-responsive participants. Finally, §4.5 discusses DICE's on-chain transaction costs.

4.1 Efficiency

This section analyzes the efficiency of our DICE algorithm in terms of per-worker and per-validator computational complexities in each learning round. A lower per-participant complexity is preferred, as it enables a wider range of honest participants to contribute meaningfully, and reduces the algorithm's overall execution latency due to the parallel execution of participants within each role.

For ease of analysis, we suppose that (i) the size of the global model is S_{GM}; and (ii) in DICE, the minimum size of the local model is $\gamma_{LM} \cdot S_{GM}$, and the minimum size of the validation model is $\gamma_{VM} \cdot S_{GM}$ (where $0 < \gamma_{VM} \leq \gamma_{LM} \leq 1$). Beyond DICE, we also analyze the current BCFL paradigm introduced in §1 for comparison.

Theorem 1. *In each round r of the current BCFL paradigm, the computational complexity of each worker is at least $\Omega(S_{GM})$ and that of each validator is at least $\Omega(|\mathbb{W}^r| \cdot S_{GM})$.*

Proof. The current BCFL paradigm requires every worker to train the full global model. Therefore, the computational complexity of every worker is at least $\Omega(S_{GM})$. Given that $|\mathbb{W}^r|$ workers produce updates in round r and the size of each worker's update is S_{GM}, each validator must correctly verify all updates to reach cross-validator consensus if it expects to receive rewards. A worker's update is typically verified by evaluating it on a validation model whose size is the same as the global model. Therefore, the computational complexity of each validator that participates honestly is at least $\Omega(|\mathbb{W}^r| \cdot S_{GM})$.

Theorem 2. *In every round r of DICE, the computational complexity of each worker is at least $\Omega(\gamma_{LM} \cdot S_{GM})$ and that of each validator is at least $\Omega(N_t \cdot \gamma_{VM} \cdot S_{GM})$.*

Proof. In DICE, every worker trains a compact local model whose size is at least $\gamma_{LM} \cdot S_{GM}$, where γ_{LM} is the ratio of local to global model size. Hence, the computational complexity of each worker is at least $\Omega(\gamma_{LM} \cdot S_{GM})$. Given updates produced by workers $w \in \mathbb{W}^r$, DICE's consensus algorithm divides the task of verifying and aggregating those updates into N_{VA} sub-tasks. To obtain rewards, each

validator must correctly perform at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated sub-tasks to generate the consensus that moves the customization task forward. Each such sub-task is either a verification (VER) or an aggregation (AGG) sub-task. A VER sub-task requires evaluating a validation model whose size is at least $\gamma_{VM} \cdot S_{GM}$, while an AGG sub-task requires aggregating two updates whose size is proportional to the validation model size (i.e., $\Theta(\gamma_{VM} \cdot S_{GM})$). Therefore, in either case, the computational complexity of a single sub-task is at least $\Omega(\gamma_{VM} \cdot S_{GM})$, and the total computational complexity of each validator is at least $\Omega(N_t \cdot \gamma_{VM} \cdot S_{GM})$.

Theorems 1 and 2 indicate that DICE can achieve much lower computational complexities than the current BCFL paradigm on both workers and validators and thus has a greater potential to enable more honest Web3 participants to contribute to customization. On the worker side, when $\gamma_{LM} < 1$ (a common scenario where engaging devices are unable to deploy the full model), the complexity is reduced from $\Omega(S_{GM})$ to $\Omega(\gamma_{LM} \cdot S_{GM})$. On the validator side, the gain is even more significant: by setting N_t to a sub-linear function of $|\mathbb{W}^r|$ (e.g., $O(\log |\mathbb{W}^r|)$ or even $O(1)$), DICE breaks the linear dependency between per-validator workload and worker scale. The optimal N_t setting will be further discussed later in §4.3.

Discussions. The preceding analysis focuses on per-worker and per-validator computational complexities. Beyond these, two additional efficiency dimensions warrant consideration: the requester-side computation overhead, and the system-level communication and coordination costs. We discuss each below.

Requester-side Computation Overhead. The model requesters in prior BCFL algorithms [50, 54, 64, 75, 77, 82, 132] are typically not involved in the iterative training and VA process after deploying smart contracts for their customization tasks, since those algorithms are designed for scenarios where all participants have relatively symmetric hardware resources. In contrast, DICE is designed for resource-asymmetric scenarios and aims to fully exploit requester-side resources to enable lightweight training and verification for hardware-constrained workers and validators.

The requester in DICE undertakes slice selection and large-to-compact model derivation. During slice selection, the requester computes importance scores for candidate slices with complexity $O(\sum_{g=1}^{G} N_{g,\text{slices}} \cdot S_{GM})$, where G ($G = 1$ under U-DPL mode) is the number of worker groups with workers in the same group sharing the same hardware capacity indicator L_w, and $N_{g,\text{slices}}$ is the number of candidate slices for group g. During large-to-compact model derivation, the requester generates emulators for both workers and validators with complexity $O(N_{\text{emulator}} \cdot S_{GM})$, where N_{emulator} is the number of emulators (each corresponding to a distinct slice among all $\mathbb{M}_w^r$ and $\mathbb{LUS}_j$).

These requester-side costs are deliberate trade-offs for enabling broader participation in customizing large PMs in Web3. The requesters typically have sufficient hardware resources to perform the above computations and can obtain a high-quality customized model and its associated high economic utility by the above computations. Moreover, the requester can periodically select slices and generate emulators (as discussed in §3.2.3), reducing computation costs over multiple rounds. Further optimization of requester-side costs via more efficient slice evaluation, model distillation, or caching strategies is a promising direction for future work.

Communication and Coordination Complexity. In addition to computation, communication and coordination overhead also affects the overall efficiency of BCFL systems. DICE coordinates all participants via a single smart contract and stores large-scale data (e.g., individual worker updates, emulators, aggregated updates, global model parameters) in an additional distributed storage system, as introduced in §2.1.2. Therefore, the total communication and coordination costs comprise two parts: on-chain interactions and off-chain data transfers.

The on-chain interaction overhead depends on the number of participants, the number of submissions each participant makes per round, and the data size of each submission. Since DICE records only lightweight data such as cryptographic hashes and file handles on the blockchain, the impact of the third factor on on-chain interaction overhead is negligible compared to those of the first two factors. Therefore, our analysis below focuses on the first two factors. The requester and each worker submit at most once per round. The requester publishes slice selection results (i.e., selected layer indexes) and the file handles and hashes of generated emulators (when selection or generation is triggered), while each worker submits the file handle and hash of its update. Therefore, the per-requester and per-worker on-chain interaction complexities are both $O(1)$. Validators interact with the smart contract in two states. In the VA_COMMIT state, each validator submits fingerprints and commitments for the sub-tasks it executes, requiring up to $O(N_{v,t}^{all})$ submissions, where $N_{v,t}^{all}$ is the total number of sub-tasks executed by validator v in this state. In the VA_REVEAL state, the file handles and hash values of all corresponding raw results can be packed into a single submission. Thus, the per-validator on-chain interaction complexity is $O(N_{v,t}^{all})$ in VA_COMMIT and $O(1)$ in VA_REVEAL. Overall, the total on-chain interaction complexity per round is $O(1 + |\mathbb{W}^r| + \sum_{v \in \mathbb{V}^r}(1 + N_{v,t}^{all}))$, with the three terms corresponding to the requester, all active workers, and all active validators, respectively.

Notably, different from some BCFL algorithms [10, 54, 82] in which each validator submits only once per round, validators in DICE submit VA results in two states: multiple times during VA_COMMIT and once during VA_REVEAL. This design is a deliberate trade-off: using more frequent but lightweight on-chain submissions in exchange for substantially lower per-validator computational complexity (from $\Omega(|\mathbb{W}^r| \cdot S_{GM})$ to $\Omega(N_t \cdot \gamma_{VM} \cdot S_{GM})$) and security against fraud attacks (proved in Theorem 3). This trade-off is favorable because off-chain computation is the primary barrier for hardware-constrained validators in customizing large-scale models. Moreover, validators can batch their commit-phase submissions to further reduce the on-chain interaction overhead in practice.

The off-chain data transfer costs are incurred when accessing the off-chain storage system. In each round, the requester uploads the emulators when emulator generation is triggered, with total size bounded by $O((|\mathbb{W}^r| + |\mathbb{V}^r|) \cdot S_{EM})$, where S_{EM} is the size of a single emulator, satisfying $S_{EM} \ll S_{GM}$ and $S_{EM} < \min(\gamma_{LM}, \gamma_{VM}) \cdot S_{GM}$. Each worker downloads its assigned slice's parameters and the corresponding emulator and uploads its update. The total data transfer volume per worker is bounded by $O(\gamma_{LM} \cdot S_{GM})$. Each validator downloads the necessary emulators and worker updates for the sub-tasks it executes and uploads the corresponding VA

results. The total data transfer volume per validator is bounded by $O(N_{v,t}^{all} \cdot \gamma_{VM} \cdot S_{GM})$. Overall, the total off-chain data transfer complexity per round is $O((|\mathbb{W}^r| + |\mathbb{V}^r|) \cdot S_{EM} + |\mathbb{W}^r| \cdot \gamma_{LM} \cdot S_{GM} + \sum_{v \in \mathbb{V}^r} N_{v,t}^{all} \cdot \gamma_{VM} \cdot S_{GM})$. All off-chain transfers are conducted through cost-effective storage systems (e.g., IPFS). Moreover, reusing emulators across rounds (as discussed in §3.2.3) can mitigate the off-chain communication and coordination overhead for both the model requester and workers/validators using those emulators.

4.2 Security

An algorithm is secure if and only if the algorithm's final results are produced by honest participants who strictly follow the algorithm's logic [56]. DICE relies on validators to reject invalid updates from malicious workers and to aggregate only qualified updates to generate final customized models. Thus, DICE is secure as long as the consensus VA results of each round are generated by honest validators. This security relies on three preconditions.

Assumption 1. Hash Collision Resistance. Given any probability threshold $\epsilon_p \in (0, 1]$, there exists a cryptographic hash function $H(\cdot)$ and an input value domain satisfying $\min(hlen, \log_2 N_{input}) > \log_2 \frac{1}{\epsilon_p}$, where *hlen* is the bit-length of H's output, and N_{input} is the cardinality of the input value domain.

Assumption 2. Sufficient Validators and Stake and Honest Majority. *The total number of validators and the total security deposit staked by all validators are both sufficiently large, i.e.,* $|\mathbb{V}_{all}| > 2\lceil \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$ *and* $E_{total} \gg E_c$. *The ratio of deposit staked by honest validators to* E_{total}, *denoted by h, is greater than* $\frac{2}{3}$.

Assumption 3. Bounded Stake Disparity. *Let* E_{min}, E_{avg} *be the minimum and average amounts of stake among all validators, respectively. We assume* $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$.

Under these assumptions, we have Theorems 3–5.

Theorem 3. *Under Assumption 1, given any threshold* $\epsilon_p \in (0, 1)$, *the probability that any malicious validator* v' *computes a* $CC_{v'}$ *value bound to its own identity in the* VA_COMMIT *state using results copied from honest validators to pass* QualificationCheck() *in the* VA_REVEAL *state of* DICE *is smaller than* ϵ_p.

Proof. For any malicious validator v', passing QualificationCheck() in the VA_REVEAL state by means of plagiarism to fraudulently claim rewards requires this validator to generate and submit a $CC_{v'}$ value bound to its own identity, i.e., $CC_{v'} = H(v'||\mathbb{VAR}_v)$, in the VA_COMMIT state, where $\mathbb{VAR}_v$ is the complete result set produced by honest validators for a specific VA sub-task. However, honest validators in the VA_COMMIT phase submit only the fingerprint (including $H(\mathbb{VAR}_v)$, $\mathbb{VAR}_v^{commit}$) and commitment ($H(v||\mathbb{VAR}_v)$), not the complete raw $\mathbb{VAR}_v$. Therefore, the only way to generate $CC_{v'} = H(v'||\mathbb{VAR}_v)$ in the VA_COMMIT state is to guess $\mathbb{VAR}_v$ from the exposed fingerprints and commitments. According to the property of the cryptographic hash function [3, 87], the probability of recovering the unknown input from the hash outputs is at most $2^{-\min(hlen,E)}$, where E is the min-entropy (in bits) of the unknown input. With $\mathbb{VAR}_v^{commit}$ submitted in the VA_COMMIT phase, $E = |\mathbb{VAR}_v \setminus \mathbb{VAR}_v^{commit}| \cdot \log_2 N_{input} \geq \log_2 N_{input}$. This indicates that, for any given threshold ϵ_p, as long as $\min(hlen, \log_2 N_{input}) > \log_2 \frac{1}{\epsilon_p}$ in Assumption 1 is satisfied, the probability that malicious validators succeed in fraud attacks is smaller than ϵ_p.

Theorem 3 indicates that it is computationally intractable for any malicious validator to succeed in fraud attacks within DICE.

In the following, we show that DICE also prevents malicious validators from successful poisoning attacks.

Theorem 4. *Under Assumptions 2 and 3, given any threshold* ϵ_p *and any* $N_t \in \mathbb{Z}^+$, *there exists a parameter* $N_c = \lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$ *such that the probability that every VA sub-task in* DICE *receives correct results from at least one honest validator is greater than* $1 - \epsilon_p$.

Proof. We prove Theorem 4 by showing that its complementary event, namely there exists a VA sub-task in DICE for which no honest validator submits correct results, occurs with probability smaller than ϵ_p.

In DICE, honest validators randomly select and execute VA sub-tasks based on their computational power and submit VA results in two states: VA_COMMIT and VA_REVEAL. We focus on analyzing the VA_COMMIT state, since honest validators will reveal raw results in VA_REVEAL that match the fingerprints and commitments they submitted in VA_COMMIT. The VA_COMMIT state will not end before (i) N_c validators have submitted results for at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated VA sub-tasks, and (ii) every sub-task has consensus results. In this state, each honest validator executes either a *conflict-related* or a *conflict-unrelated* sub-task.

For conflict-related sub-tasks, including controversial sub-tasks and their descendants, Theorem 5 guarantees that the consensus results of every controversial sub-task are produced by honest validators with probability greater than $1 - \epsilon_p$. Moreover, DICE requires every controversial sub-task's result proposer to submit results for all descendants of that controversial sub-task. This design together with Theorem 5 necessarily implies that, given any conflict-related sub-task, the probability that no honest validator submits correct results for this sub-task is smaller than ϵ_p.

Below, we show that with $N_c = \lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$, the probability that no honest validator submits correct results for any given conflict-unrelated sub-task is smaller than ϵ_p.

During the execution time of VA_COMMIT, each honest validator randomly selects at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated sub-tasks for which it submits results, from the VA task forest. This forest is composed of N_{VA} sub-tasks, where the total number of conflict-unrelated sub-tasks is $N_{VA} - N_{CF}$ and $N_{VA} = (2|\mathbb{W}^r| - 1)N_{lus}$. Therefore, the probability that all honest validators do not submit results for a specific conflict-unrelated sub-task, denoted by Pr, is given by Eq. 12, where $\mathbb{V}_h$ is the set of honest validators who have submitted results for at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated sub-tasks.

$$Pr = (1 - \frac{\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil}{N_{VA} - N_{CF}})^{|\mathbb{V}_h|} \leq (1 - \frac{N_t}{(2|\mathbb{W}^r| - 1)N_{lus}})^{|\mathbb{V}_h|} \quad (12)$$

Let $N_{malicious}$ denote the total number of malicious validators. It is easy to derive that $N_{malicious} \leq \frac{(1-h)E_{avg}|\mathbb{V}_{all}|}{E_{min}}$. Based on $h > \frac{2}{3}$, $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, we have $N_{malicious} < \frac{|\mathbb{V}_{all}|}{2}$. Then, we obtain Eq. 13–14.

$$|\mathbb{V}_h| \geq N_c - N_{malicious} > N_c - \frac{|\mathbb{V}_{all}|}{2} \quad (13)$$

$$Pr < (1 - \frac{N_t}{(2|\mathbb{W}^r| - 1)N_{lus}})^{N_c - \frac{|\mathbb{V}_{all}|}{2}} \quad (14)$$

With $N_c = \left\lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \right\rceil$, we have $Pr < \epsilon_p$. The above analysis proves that the overall probability that there exists any VA sub-task (whether conflict-related or conflict-unrelated) for which no honest validator submits correct results is smaller than ϵ_p, indicating the correctness of Theorem 4.

Theorem 4 states that all VA sub-tasks in DICE will receive results from honest validators with high probability. This directly yields the following corollaries.

Corollary 1. *Under the same conditions as Theorem 4, if a VA sub-task receives a poisoned result from a malicious validator, the probability that the sub-task becomes controversial is greater than* $1 - \epsilon_p$.

Corollary 2. *Under the same conditions as Theorem 4, for every non-root and non-controversial sub-task in* DICE*, the consensus results are produced by honest validators with probability greater than* $1 - \epsilon_p$.

Then, we show that all root and controversial sub-tasks will finally be resolved by honest validators.

Theorem 5. *Under Assumption 2, given any threshold* ϵ_p*, there exist parameters* $E_c = 18Z^2_{1-\epsilon_p}$ *and* $\epsilon_c = \frac{1}{2}(1 + Z_{1-\epsilon_p}\sqrt{\frac{1-h}{E_c}} + Z_{\epsilon_p}\sqrt{\frac{h}{E_c}})$ *such that the probability that consensus results for every root or controversial sub-task in* DICE *are produced by honest validators is greater than* $1 - \epsilon_p$*, where* Z_{ϵ_p} *denotes the* ϵ_p*-quantile of the standard normal distribution, and* $Z_{1-\epsilon_p} = -Z_{\epsilon_p}$.

Proof. For any root or controversial sub-task, DICE adopts committee voting [56, 126] to determine the consensus. Only results that receive more than $E_c \cdot \epsilon_c$ votes will be accepted as consensus results. As proved in prior work [56], under Assumption 2, for any small threshold ϵ_p, setting $E_c = 18Z^2_{1-\epsilon_p}$ and $\epsilon_c = \frac{1}{2}(1 + Z_{1-\epsilon_p}\sqrt{\frac{1-h}{E_c}} + Z_{\epsilon_p}\sqrt{\frac{h}{E_c}})$ ensures that the consensus results are produced by honest validators with a probability greater than $1 - \epsilon_p$. This indicates the correctness of Theorem 5.

Theorems 4 and 5 collectively yield the following corollary, proving that the probability that any malicious validator succeeds in poisoning attacks in DICE is negligible.

Corollary 3. *Under the same conditions as Theorems 4 and 5, all VA sub-tasks in* DICE *have their consensus results produced by honest validators with probability greater than* $1 - \epsilon_p$.

Corollary 3 and Theorem 3 jointly prove that DICE prevents malicious validators from successful poisoning and fraud attempts with overwhelming probability, ensuring that only honest validators produce consensus results and the complete customization task is secure.

Discussion. The correctness of Theorems 3–5 relies on Assumptions 1–3. DICE can adopt the following designs to ensure these assumptions hold.

Designs supporting Assumption 1. To satisfy the inequality in Assumption 1, i.e., $\min(hlen, \log_2 N_{input}) > \log_2 \frac{1}{\epsilon_p}$, we first set the parameter ϵ_p as a system parameter that cannot be changed after the system is initialized. We recommend $\epsilon_p = 10^{-12}$ to ensure a negligible probability for successful attacks. Then, we select SHA-256 as the cryptographic hash function used in the algorithm runtime, which provides an output length of 256 bits, and use uint256 as the type of data packed in the result set $\mathbb{VAR}_v^{reveal}$ that each honest validator does not submit in the VA_COMMIT state. These settings satisfy that the hash length, *hlen*, and the bit length of the input domain, $\log_2 N_{input}$, both equal to 256, are greater than $\log_2 \frac{1}{\epsilon_p} \approx 40$ for $\epsilon_p = 10^{-12}$.

Designs supporting Assumptions 2–3. To ensure the following three critical security preconditions hold: (i) sufficient validators and total stake, $|\mathbb{V}_{all}| > 2\lceil \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$, $E_{total} \gg E_c$, (ii) an honest majority, $h > \frac{2}{3}$, (iii) bounded stake disparity, $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, we adopt a carefully designed staking scheme. This scheme requires each validator to stake between $E_{\min}$ and $\frac{3}{2}E_{\min}$ units of cryptocurrency as a security deposit when joining a customization task and sets $E_{\min} = \max(E_c, VR + \epsilon_{penalty})$. Given any customization task, only after the registration of $3\lceil \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$ validators will DICE start the task. At runtime, DICE dynamically removes the validators whose staked deposit falls outside the required range $[E_{\min}, \frac{3}{2}E_{\min}]$. If the number of validators meeting the deposit requirement drops below $3\lceil \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \epsilon_p \rceil$, DICE then restarts participant registration to replenish the validator pool. These designs collectively ensure a sufficiently large stake and enforce the bounded stake disparity. As a consequence of $E_{total} \gg E_c$ and $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, it becomes intractable for malicious validators to control more than one-third of the total stake, which ensures $h > \frac{2}{3}$. Another benefit of $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$ is that it prevents an excessively large deposit from inflating the minimum number of validators required for security (derived from Theorem 4), thereby controlling the difficulty of recruiting sufficient validators.

In addition to the above designs, DICE further encourages validators to behave honestly (thus ensuring $h > \frac{2}{3}$) by locking each validator's security deposit and rewards in smart contracts for a certain period. Suppose that a validator v stakes cryptocurrency (or earns a positive reward) in round r of a customization task T. DICE will lock that cryptocurrency along with any rewards earned in that round until this validator completes at least $N_t \cdot \epsilon_{lock}$ new VA sub-tasks after round r in task T (or subsequent customization tasks) and is not detected as malicious. Here, $\epsilon_{lock} \in \mathbb{Z}^+$. Under this design, validators, particularly those holding a large share of the total stake, are incentivized to stay honest when choosing between honest behavior and deviating from the algorithm to deceive the model requester. If choosing the former, validators will be rewarded for their contributions. Otherwise, the model requester may cease submitting new customization tasks. Then, the validator's staked deposit risks being locked indefinitely without reward or burned upon detection of malicious activity. Consequently, staying honest is the better choice, as it yields higher expected profit than deviating from the algorithm logic.

4.3 Per-Validator Workload Reduction and Scalability Gains versus System Overhead

Theorems 2 and 4 reveal a trade-off between two desirable but conflicting goals. We now discuss this trade-off and navigate it by setting N_t.

- **Goal 1: Minimize per-validator computational complexity (favoring smaller N_t as shown in Theorem 2).** This complexity directly determines: (i) *Individual validator overhead and*

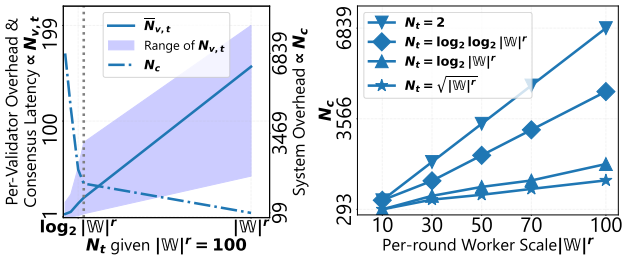


Figure 5: Effects of varying N_t, given $\epsilon_p = 10^{-12}$, $N_{lus} = 1$.

participation barrier: Lower complexity makes it easier for honest participants to contribute as validators. (ii) *Cross-validator consensus latency*: Since validators work in parallel and pipeline off-chain computation of VA sub-tasks without waiting for each sub-task's on-chain confirmation, this latency is dominated by $N_{v,t} \times \bar{t}_v$, where $\bar{t}_v$ is v's average time to execute one sub-task. (iii) *System scalability w.r.t. workers*: When per-validator complexity is independent of worker scale (achieved by setting N_t to $O(1)$), this scalability is maximized.

- **Goal 2: Minimize the required validator scale (favoring larger N_t, as quantified in Theorem 4)**. Reducing N_t leads to a super-linear increase in the number of validators required for security (N_c). A larger N_c brings three issues: (i) *Higher recruitment difficulty*: In ecosystems with limited participants, recruiting N_c or more validators may become infeasible. (ii) *Increased system-level overhead*: Since the complexities of on-chain interaction and off-chain data transfer in each round are both linear to $|\mathbb{V}^r|$ as analyzed in §4.1 and $N_c \leq |\mathbb{V}^r|$ must hold to reach consensus, the system coordination and communication complexities both increase linearly with N_c. (iii) Potential offset of latency benefit. Slower validators with larger $\bar{t}_v$ may be introduced, increasing the maximum $N_{v,t} \times \bar{t}_v$, though this effect is limited in practice by the actual $\bar{t}_v$ distribution and the reduction of $N_{v,t}$ with smaller N_t.

The trade-off between these two goals reflects a common pattern in distributed systems: reducing per-node workload to improve individual efficiency and support larger-scale lower-barrier participation often incurs higher system-level burden [17, 34]. Prior BCFL systems did not encounter this trade-off since they never reduced per-validator complexity [50, 54, 64, 64, 75, 77, 82, 132], leaving a critical gap: how to scale customization to hundreds or more workers without assuming all validators possess enterprise-grade hardware.

DICE navigates this trade-off by using N_t as a tuning knob between the above goals. Fig. 5(left) simulates the effects of varying N_t under fixed worker scale. Fig. 5(right) depicts the growth of N_c with increasing workers under different N_t. For most scenarios, we recommend $N_t = \lceil \log |\mathbb{W}^r| \rceil$ as a balanced choice, which achieves: (i) substantially lower per-validator complexity than the current BCFL paradigm (from linear to *sub-linear* in $|\mathbb{W}^r|$), reduced participation barrier, and improved scalability; (ii) *moderate* growth rate on required validator scale and *manageable* system overhead; (iii) *provable* security. DICE offers users the flexibility to adapt N_t to actual deployment constraints such as available participant scale, tolerable system overhead, security requirements, and individual resource budgets.

4.4 Impact of Non-responsive Participants

Below, we discuss the impact of non-responsive participants.

4.4.1 Impacts on Liveness. The customization task may stall in TRAINING, VA_COMMIT, or VA_REVEAL state, if too many participants become non-responsive such that the number of active workers or validators falls below the required thresholds. DICE inherits a non-responsive participant handling scheme from prior work [56], which sets dynamically adjusted timeouts for those states, to prevent deadlock (see §2.3.3). Under the asynchronous network model with messages eventually delivered although delays are unbounded, liveness is guaranteed as long as at least one honest participant remains active and has access to the task's smart contract.

4.4.2 Impacts on Efficiency and Model Utility. DICE's overall execution time could increase when non-responsive participants slow down the update collection or the consensus process, or could decrease due to early termination invoked by the non-responsive participant handling scheme. DICE's model performance may degrade because fewer valid updates are available per round when a growing number of honest workers become non-responsive, or because early termination leads to underfitting. Since DICE inherits the non-responsive participant handling scheme from prior work [56], these impacts are similar to those analyzed in that work, and we refer readers to that work for a detailed experimental evaluation of these impacts.

We note that non-responsive participants do not affect the per-participant computational complexity bounds derived in §4.1. These bounds measure individual-level efficiency performance, determined by resource constraint parameters (e.g., L_w, L_v) reported when participants join the customization task or model scale (e.g., S_{GM}) specified by the model requester.

4.4.3 Impacts on Scalability. The greater the number of non-responsive participants, the smaller the pool of available workers and validators, making it harder to meet the required participant scale. DICE mitigates this impact from two directions. On the one hand, DICE's consensus protocol is designed to operate in permissionless blockchain environments, where any device with Internet access can join as a worker or validator without prior authorization. This design choice maximizes the potential participant pool and inherently improves scalability by allowing a larger and more diverse set of participants, thereby increasing resilience to non-responsive participants. On the other hand, as discussed in §4.3, the tunable parameter N_t allows the system to adapt to the actual available participant scale, further mitigating non-responsive participant impacts.

4.4.4 Impacts on Security. As established in §4.2, DICE is secure as long as consensus VA results are produced by honest validators, which depends on Assumptions 2–3 (sufficient validators and stake, honest majority, bounded stake disparity). If too many honest validators are non-responsive, these assumptions may be temporarily violated. In such cases, DICE replenishes the validator pool until

assumptions are restored, and then resumes the customization process. This means that, whenever DICE makes progress, the security assumptions hold, and the algorithm remains safe.

4.5 On-chain Transaction Cost

The overall economic costs of customizing a pretrained model in a typical BCFL system mainly comprise two parts: off-chain computation costs and on-chain transaction costs. DICE focuses on the former for hardware-constrained workers and validators, reducing their per-participant per-round computational complexity through DPL, MSV, TCC, and EDI (as analyzed in §4.1). Although DPL and MSV involve off-chain computation for slice selection and emulator generation, this overhead is borne by the requester, who initiates the task and is typically assumed to possess sufficient hardware resources. For on-chain transaction costs, DICE incorporates the following designs. Since BCFL systems can be deployed on various underlying blockchain platforms with different fee models, we use Ethereum [16] as a representative example for discussion.

Ethereum uses gas as a unit to measure the computational and storage resources required to perform an on-chain procedure. The transaction fee for executing an on-chain procedure is calculated as: $fee = gas_used \times gas_price$, where *gas_used* is the total amount of gas consumed by the smart contract's EVM instructions and data storage, and *gas_price* is the market-determined price per unit of gas. Following previous work [56], DICE focuses on optimizing *gas_used*, as *gas_price* is determined by the market rather than by BCFL algorithm design.

DICE reduces the on-chain gas costs by two key designs. First, large-scale data such as a worker's updates or aggregated updates are stored in an additional cost-effective system such as IPFS rather than on the blockchain, with only cryptographic hashes and file handles persisted on-chain. This makes *gas_used* largely independent of the model size, avoiding the substantial gas overhead incurred by prior BCFL systems that perform on-chain aggregation of large model parameters [2, 109, 120]. Second, DICE applies gas-efficient smart contract coding practices [73, 76], such as using low-cost EVM instructions and batching multiple operations into single transactions, to further reduce the gas costs of contract execution. According to previous work [56], the above designs can ensure that on-chain gas fees account for only a small fraction of the total costs, compared to off-chain computation costs, which is indispensable for engaging a wider range of honest (but potentially hardware-constrained) participants and customizing an effective model.

5 Experiments

We structure a comprehensive evaluation of DICE into five parts to systematically assess its performance under varying conditions. §5.2 evaluates DICE with *honest but hardware-constrained* participants, against an ideal but infeasible scenario where all participants are honest and hardware-unlimited, establishing DICE's performance baseline. §5.3 compares DICE against the current BCFL paradigm in realistic scenarios involving *both hardware-constrained and malicious* participants, demonstrating DICE's holistic advantage. §5.4 studies DICE's flexibility in integrating with existing efficient customization techniques, which, coupled with adjustable N_t discussed in §4.3, enables DICE to accommodate diverse device constraints and model performance requirements. §5.5 extends the evaluation to larger-scale PMs to demonstrate DICE's generalizability beyond the models used in the main experiments. Finally, §5.6 further studies the contribution of DPL and MSV schemes for effective lightweight training and verification and the necessity of using TCC with EDI for efficient consensus.

Table 3: Task settings. ‡-marked tasks involve further pretraining before fine-tuning; others fine-tune the PM directly.

Settings	Vision Tasks	Language Tasks
Type	Image Classification	NER or RE
Dataset	FedISIC [1] ‡, CIFAR100 [91], DermaMNIST [117, 118]	2018n2c2 [48] ‡, BC5-disease (BC5-d) [48], JNLPBA [46], GAD [48], DDI[36], ChemProt[95]
Model	ViT-base (default), DINOv2-large	BERT-base (default), Qwen2.5-0.5B

5.1 Experimental Settings

Our experiments are conducted on a private Ethereum testnet deployed on our in-house private cloud to avoid the real economic costs introduced by invoking smart contracts and making transactions on the public Ethereum mainnet. We follow prior work [56] to configure the blockchain network and refer readers to that work for more details. Below, we introduce the configuration of customization tasks, participants, the BCFL baselines used for comparison, and evaluation metrics.

Task Settings. Our experiments are conducted on nine customization tasks summarized in Table 3. The vision tasks customize ViT [110] for image classification on three datasets commonly used in federated learning research [1, 8, 60, 70, 81, 103, 122, 123]. The language tasks customize BERT [25, 107] for named entity recognition or relation extraction on six biomedical domain datasets from prior work [32, 48, 79, 80]. The extended experiments in §5.5 use larger-scale models, namely DINOv2 [78] for vision tasks and Qwen2.5 [97] for language tasks. For both vision and language tasks, we select one biomedical dataset to evaluate the performance of fine-tuning the PM on that dataset with a prior further-pretraining step. We use MedMNIST [117, 118], a large-scale collection of standardized biomedical images, and PubMed [32], a large-scale biomedical text dataset, for further-pretraining in vision and language tasks, respectively.

Participant Configuration. We deploy 100 workers and 1641 validators for vision customization tasks, and 50 workers and 1020 validators for language customization tasks, along with a model requester who submits all customization tasks. These worker scale settings follow prior work [18, 56, 85, 92]. For language tasks, we use a smaller number of workers due to the higher computational costs of customizing BERT and Qwen2.5 compared to ViT and DINOv2. Given worker scale settings, we compute the required number of validators using the formula derived in §4.2, the N_t setting recommended in §4.3, and a relaxed security threshold $\epsilon_p = 10^{-2}$ that can reduce the required validator scale to achieve a balance between experimental costs and reproducibility.

Each participant runs in a dedicated virtual machine hosted in our in-house private cloud. We associate each participant with a unique Ethereum testnet account. Each worker is allocated a subset

Table 4: Worker and validator memory overhead (MC_w, MC_v in GB) for model training and multi-update ($|\mathbb{W}^r|$-update) verification when customizing Qwen2.5-0.5B. S_{LM}/S_{VM} denotes local/validation model size in GB. Measurement environment follows Table 1.

L_w/L_v	S_{LM}/S_{VM}	MC_w	Hardware Req.	MC_v	Hardware Req.
24	1.8	16.4	RTX 4090 24GB ↑	122.2	2× H100 80GB ↑
20	1.6	13.8	RX 9070 XT 16GB ↑	108.3	2 × H100 80GB ↑
16	1.4	11.2	RTX 5070 12GB ↑	94.5	RTX Pro 6000 96GB ↑
12	1.2	8.6	ARC B570 10GB ↑	80.6	2×RTX A6000 48GB ↑

of training samples from the training datasets of the customization tasks. To simulate the non-IID data distribution prevalent in real-world cases, we allocate sample sizes across workers using a Dirichlet distribution and differentiate label distributions across workers based on the number of unique labels in each dataset. Validators have access to a held-out validation dataset to evaluate worker updates. For datasets that include dedicated validation samples, we use them directly. In cases where such samples are unavailable, we split the test set in half, using one half for validation and the other for final model evaluation.

Unless otherwise stated, participants in each experiment are heterogeneous in hardware resources and potentially malicious.

We simulate hardware constraints by limiting the number of H-layers deployable on workers and validators. The number of deployable H-layers on each worker w is set as $L_w = \lfloor \frac{L_G}{2} + \frac{L_G}{2} \cdot \left(1 - \frac{|\mathbb{D}_w| - D_{min}}{D_{max} - D_{min}}\right) \rceil$, where D_{max} and D_{min} are the maximum and minimum sizes of a worker's training dataset, respectively. This simulates a scenario where workers with abundant task-specific data cannot deploy a full model due to hardware limitations. For validators, we simulate hardware heterogeneity by sampling their deployable H-layer counts from a normal distribution: $L_v \sim \mathcal{N}\left(\frac{3L_G}{4}, \frac{L_G^2}{144}\right)$, clipped to the range $[L_G/2, L_G]$. This reflects a common heterogeneous scenario where some validators can deploy nearly full models, while others are constrained and even unable to deploy a worker's complete update when $L_{vm} < L_{wm}$. Table 4 takes Qwen2.5-0.5B, the largest PM used in our experiments, as an example to illustrate the mapping from the total number of H-layers in a pretrained model to actual GPU memory overhead and hardware requirements. This table indicates that configuring deployable H-layers provides a controllable and reproducible proxy scheme to simulate real-world hardware constraints and heterogeneity.

Regarding adversarial settings, we designate 20% of workers and 20% of validators as malicious, with the remaining participants being honest. An exception is the experiment in §5.2, where we configure all participants as honest to establish DICE's performance baseline. Each malicious participant randomly chooses between poisoning and fraud in each round. For any malicious worker, if it chooses to launch fraud attacks, it copies an honest worker's update to illegitimately claim rewards. Otherwise, it submits updates trained on mislabeled backdoor examples for poisoning attacks [7, 12]. A malicious worker cannot launch both attacks in a single round, as DICE permits only one update per worker per round. For any malicious validator, if it chooses to launch fraud attacks, it submits the valid-looking but fraudulent commitments, fingerprints, and raw VA results copied from honest validators. Otherwise, it submits poisoned VA results, generated by assigning random scores above the discard threshold to each malicious worker's update. All validators stake the same amount of security deposit in our experiments, so the fraction of total stake controlled by malicious validators equals their share of the total validator count.

BCFL baseline Configuration. We implement the current BCFL paradigm introduced in §1 based on a representative system, namely Refiner [56, 130]. This choice is justified for two reasons. First, Refiner shares the same participant resource requirements as mainstream BCFL algorithms [24, 28, 54, 86, 108], namely requiring each worker to train a full model and each validator to fully verify every worker's update. This allows Refiner to faithfully represent the computational burden of the current BCFL paradigm. Second, Refiner is designed under the same threat model as DICE, addressing both poisoning and fraud attacks. In contrast, many BCFL algorithms [20, 24, 28, 44, 54, 59, 69, 84, 86, 108] neglect fraud attacks that have a negligible impact on model performance, making them unsuitable for a direct comparison under our broader threat model. Owing to the above characteristics, Refiner enables a fair and meaningful comparison of the current BCFL paradigm with DICE.

Our experiments also implement several enhanced variants of the current BCFL paradigm powered by four efficient customization techniques. (i) *LoRA* [40], the most popular PEFT method adopted in federated learning research [94, 113, 131], which exemplifies the family of methods that insert small trainable modules while freezing the backbone of the original pretrained model. (ii) *BitFit* [11], a PEFT approach that does not alter the model architecture but only tunes a subset of selected parameters. Together with LoRA, BitFit covers two fundamentally different types of PEFT techniques discussed in §6 without redundancy. (iii) *Prune* [41, 61, 112] and (iv) *KD* [37, 43, 89, 111], mainstream model compression techniques applicable to both model training and inference scenarios. For the KD-enhanced variants, we use DeiT [99] as the compressed version of ViT and DistilBERT [89] as the compressed version of BERT, respectively. In all baselines, the embedding layers of PMs are frozen.

Evaluation Metrics. The evaluation metrics used in our experiments include (i) Efficiency Metrics: S_{LM} (GB), the minimum size of local models deployed on the workers, $|\theta_w^r|$ (GB), the maximum size of parameters to be trained on the minimum local model, S_{VM} (GB), the minimum size of validation models deployed on the validators, N_{VM}, the minimum number of validation models that an honest validator must load for consensus generation. Based on empirical results from previous work [31, 38, 47, 72, 105] and Table 4, S_{LM} and $|\theta_w^r|$ jointly determine the worker's training overhead (including forward/backward computation, activation and gradient memory, and communication), while $S_{VM} \times N_{VM}$ determines the validator-side costs for consensus generation. These metrics enable consistent comparisons across different hardware platforms and thus are selected over hardware-dependent measurements such as peak GPU memory or actual execution time to enhance reproducibility. (ii) Effectiveness Metrics: *ACC* and *F*1, the accuracy and F1 score of the customized global model on downstream tasks. (iii) Security Metrics: *RMW* and *RMV*, the proportion of total rewards successfully claimed by malicious workers and validators, respectively.

Table 5: Results of handling constrained workers (upper half) and constrained validators (lower half). Top-1 and top-2 results are marked in bold and underlined, respectively.

Methods	$\mathbb{W}_{all}$	$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	ACC ↑			$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	F1 ↑			$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	F1 ↑		
		Vision Tasks		FedISIC	CIFAR100	DermaMNIST	NER Tasks		n2c2	JNLPBA	BC5-d	RE Tasks		DDI	GAD	ChemProt
BCFL		100.0%	100.0%	79.5%	94.0%	82.9%	100.0%	100.0%	90.7%	98.5%	89.5%	100.0%	100.0%	44.3%	92.7%	13.8%
+LoRA	✗	101.6%	1.7%	75.7%	91.5%	78.3%	101.2%	1.6%	86.4%	93.5%	85.2%	101.2%	2.2%	43.8%	87.4%	13.1%
+BitFit		100.0%	**0.2%**	71.5%	94.8%	75.2%	100.0%	**0.1%**	81.6%	59.3%	84.5%	100.0%	**0.8%**	42.2%	76.3%	23.0%
+Prune	✓	50.4%	50.0%	95.9%	90.9%	99.6%	**60.9%**	50.0%	97.4%	98.2%	98.5%	**61.1%**	50.4%	93.3%	99.6%	78.8%
+KD		**3.3%**	3.1%	91.7%	80.3%	91.2%			97.1%	99.0%	**99.7%**			92.8%	**99.9%**	97.6%
DICE	✓	50.4%	41.6%	**98.5%**	**98.5%**	**99.8%**	**60.9%**	41.7%	**98.5%**	**99.2%**	98.8%	**61.1%**	42.1%	**96.6%**	99.7%	**99.5%**

Methods	$\mathbb{V}_{all}$	$S_{VM}\downarrow$	$N_{VM}\downarrow$	ACC ↑			$S_{VM}\downarrow$	$N_{VM}\downarrow$	F1 ↑			$S_{VM}\downarrow$	$N_{VM}\downarrow$	F1 ↑		
		Vision Tasks		FedISIC	CIFAR100	DermaMNIST	NER Tasks		n2c2	JNLPBA	BC5-d	RE Tasks		DDI	GAD	ChemProt
BCFL				**100.0%**	**100.0%**	**100.0%**			**100.0%**	**100.0%**	**100.0%**			**100.0%**	**100.0%**	**100.0%**
+LoRA	✗	100.0%		97.2%	98.4%	97.5%	100.0%		96.3%	96.0%	97.2%	100.0%		96.9%	97.3%	94.9%
+BitFit			100.0%	89.9%	92.0%	90.7%		100.0%	91.0%	93.3%	87.5%		100.0%	90.1%	92.3%	89.7%
+Prune	✓	50.4%		95.9%	90.9%	99.6%	**60.9%**		97.4%	98.2%	98.5%	**61.1%**		93.3%	99.6%	78.8%
+KD		**3.3%**		91.7%	80.3%	91.2%			97.1%	99.0%	99.7%			92.8%	99.9%	97.6%
DICE	✓	50.4%	**3.5%**	98.1%	99.4%	99.8%	**60.9%**	**6.0%**	98.1%	99.2%	98.8%	**61.1%**	**6.0%**	97.6%	98.2%	98.7%

Below, we run each experiment three times with different random seeds and report the average of the results of the three runs. The major results are presented as percentages relative to **an ideal scenario**, where all participants are honest and unconstrained: each worker trains the full model, and each validator executes the complete VA tasks.

5.2 Efficiency Performance

We first evaluate the efficiency of DICE with honest yet hardware-constrained participants against the ideal scenario without constrained participants. To isolate the impact of each participant role's constraints, we conduct two controlled experiments: one with only the workers constrained and another with only the validators constrained. Table 5 reports results of the two experiments and reveals the following findings:

(i) Compared to the current BCFL paradigm, DICE reduces worker model size (S_{LM}) by 40–50% and tunable parameters ($|\theta_w^r|$) by nearly 60%, enabling a substantial reduction in worker-side computational and memory costs. It also reduces validator-side model scale (S_{VM}) by 40–50% and the number of validation models that each validator must deploy in the consensus process (N_{VM}) by an order of magnitude. Consequently, the product $S_{VM} \times N_{VM}$, which governs validator costs, becomes substantially smaller. Additionally, DICE maintains $\geq$ 97% of ideal performance across tasks under both worker-constrained and validator-constrained experiments.

(ii) BCFL and its PEFT-enhanced variants (+LoRA, +BitFit) sacrifice over 10% of ideal performance in most tasks of worker-constrained experiments, as they cannot facilitate workers who are unable to deploy a full model to contribute their task-specific data. Under validator-constrained experiments that remove worker constraints to enable usage of all data, these baselines achieve better ACC/F1. However, these baselines require not only full model deployment and forward computation but also multi-update verification, excluding honest but hardware-constrained participants from engaging as validators for more robust system security.

(iii) Compression-enhanced variants (+Prune, +KD), though effective at enabling constrained participants to engage in customization, suffer 5–20% ACC/F1 drops on complex tasks (e.g., FedISIC, CIFAR100, DDI). The root cause is that the final models customized by these baselines have significantly fewer parameters than the original PMs, which limits their representational capacity for tasks with complex data distributions.

In summary, DICE reduces costs on both workers and validators, lowering the participation barrier while achieving near-ideal model performance. On the worker side, DICE addresses the inherent limitations of compared baselines by DPL. DPL eliminates the full model deployment requirement on constrained participants without compromising model structure on requesters, and adjusts the training pattern dynamically, thus yielding reliable lightweight training across diverse tasks. On the validator side, DICE's efficiency gain is more pronounced. Using MSV, TCC, and EDI, DICE not only reduces per-update verification costs but also resolves a critical scalability bottleneck that prior work cannot address.

5.3 Comparison with Current BCFL Paradigm

We then compare DICE with the current BCFL paradigm and its enhanced variants under a realistic scenario that concurrently incorporates hardware-constrained and malicious participants.

Fig. 6 shows how the performance of PMs customized by different baselines varies as participant costs accumulate. DICE achieves a superior performance frontier, attaining near-ideal ACC/F1 at significantly lower costs than BCFL and its PEFT-enhanced variants, while avoiding the apparent performance degradation observed in compression-enhanced variants on complex tasks (e.g., +Prune on CIFAR100, DDI, ChemProt; +KD on FedISIC, GAD).

Fig. 7 zooms in on the results to examine how DICE addresses the limitation of baselines through four synergistic techniques. The first sub-figure shows that, via DPL and MSV, DICE drastically reduces the size of the model that each worker or validator loads for producing or verifying an update. The second sub-figure shows that DICE significantly reduces the number of validation models that an honest validator must load in the consensus process by TCC and EDI. Together, the above two sub-figures explain DICE's slower rate of cost accumulation on constrained participants observed in Fig. 6. Finally, the last sub-figure shows that DICE achieves full reward protection, on par with the BCFL paradigm, meaning that poisoning

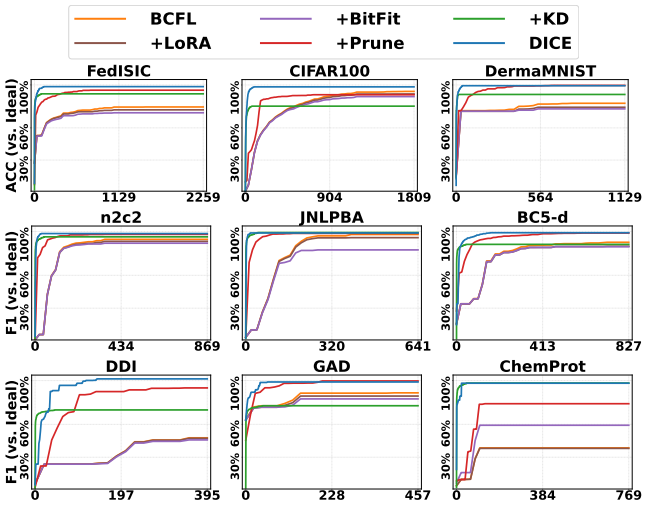


Figure 6: Performance of customized model under different baselines. X-axis: total size of models that the most hardware-constrained worker and validator have loaded at runtime (participant runtime costs $\propto S_{LM} + N_{VM} \cdot S_{VM}$).

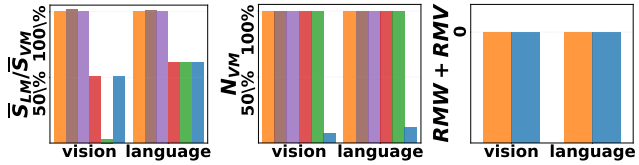


Figure 7: Per-participant workload (S_{LM}, S_{VM}, N_{VM}) and security performance ($RMW + RMV$) under different baselines.

and fraud attempts are successfully thwarted. This indicates that DICE's near-ideal model performance and efficiency gains observed in Fig. 6 do not come at the cost of security.

Collectively, results in Fig. 6–7 not only further corroborate the findings in §5.2, but also confirm that DICE establishes a practical paradigm that simultaneously controls participant expenses, lowers participation barriers, maintains model utility and robust security for customization tasks in real-world Web3 applications.

5.4 Flexibility Performance

Previous experiments show that existing efficient customization techniques, despite having limitations, still offer practical value. For example, PEFT reduces the number of parameters to be trained, while compression achieves competitive accuracy with smaller local/validation models on certain tasks. Therefore, we conduct additional experiments to study DICE's compatibility with these techniques: (i) using PEFT techniques (LoRA, BitFit) together with DPL, and (ii) replacing DPL with compression techniques (Prune, KD). DICE's validator-side designs remain unchanged.

Table 6 shows the results of this experiment and reveals three key findings. First, PEFT techniques can further lower backward computation overhead on workers with reduced update size $|\theta_w^r|$, but they do not reduce worker-side or validator-side model size S_{LM}, S_{VM}, and thus offer little benefit to participants who cannot deploy

Table 6: Results of DICE integrated with existing efficient customization techniques on different tasks.

Methods	$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	$S_{VM}\downarrow$	$N_{VM}\downarrow$	ACC/F1 ↑		
					FedISIC	CIFAR100	DermaMNIST
DICE	50.4%	41.6%	50.4%		98.3%	97.9%	99.1%
+LoRA	51.0%	0.6%	50.4%		94.0%	94.3%	94.6%
+BitFit	50.4%	0.1%	50.4%	3.5%	89.1%	89.2%	89.7%
+Prune	50.4%	50.0%	50.4%		96.1%	90.9%	98.6%
+KD	3.3%	3.1%	3.3%		91.9%	80.1%	91.2%
					n2c2	JNLPBA	BC5-d
DICE	60.9%	41.7%			97.9%	98.7%	98.1%
+LoRA	61.4%	0.6%			94.1%	94.6%	95.9%
+BitFit	60.9%	0.1%	60.9%	6.0%	91.4%	91.7%	93.7%
+Prune	60.9%	50.0%			97.3%	96.9%	97.6%
+KD	60.9%	50.0%			97.1%	98.6%	98.7%
					DDI	GAD	ChemProt
DICE	61.1%	42.1%			96.1%	98.6%	98.3%
+LoRA	61.6%	0.6%			91.1%	94.7%	94.0%
+BitFit	61.1%	0.1%	61.1%	6.0%	90.3%	91.8%	90.8%
+Prune	61.1%	50.4%			92.4%	97.8%	79.4%
+KD	61.1%	50.4%			92.0%	98.9%	97.1%

or perform forward passes on the full model. Second, compression techniques provide alternative paths to simultaneously reduce S_{LM}, $|\theta_w^r|$, and S_{VM}. An extreme reduction is achieved on vision tasks by KD, which uses a much smaller pretrained model, DeiT-Tiny, as a substitute for the requested ViT. Finally, the efficiency gains from these techniques come with trade-offs in model performance, which vary across tasks and methods.

Based on these findings, DICE offers two configuration options. The first option is to use its DPL and MSV schemes with PEFT, extreme compression, or other compatible customization techniques (see related work in the technical report [58]) to further reduce the overhead of producing and verifying an update, at the cost of potential model utility degradation. The second option is to use the default DPL and MSV to balance efficiency and model performance. Both options work alongside our core innovations, TCC and EDI, which enable BCFL systems to scale to more workers while maintaining security. Together with the adjustable parameter N_t in TCC, these options enable DICE to accommodate diverse device constraints and model performance requirements in practice.

5.5 Customizing Dinov and Qwen

This section further evaluates DICE's generalizability by customizing larger-scale pretrained models than those used in the main experiments, including DINOv2 [78] for vision tasks and Qwen2.5 [97] for language tasks. We also report the results of the current BCFL paradigm for reference.

Figs. 8–9 present the results of this experiment. The results consistently reproduce key findings from § 5.3. DICE achieves a superior performance frontier compared to the current BCFL paradigm, attaining higher accuracy/F1 at significantly lower participant costs across different tasks. DICE's four synergistic techniques, DPL, MSV, TCC, and EDI, continue to effectively reduce the model size loaded by workers and validators, decrease per-validator verification workload, and maintain full reward protection. These consistent results across different model families demonstrate that DICE's efficiency gains and security guarantees are model-agnostic, reinforcing its

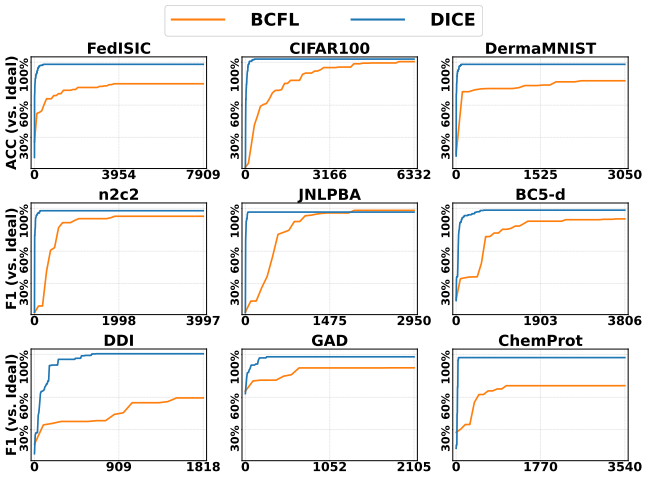


Figure 8: Performance of DINOv2 and Qwen2.5 customized by different baselines. X-axis follows Fig. 6.

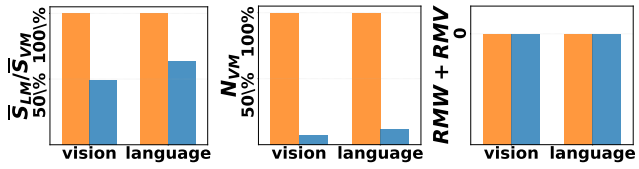


Figure 9: Per-participant workload (S_{LM}/S_{VM}, N_{VM}) and security performance ($RMW + RMV$) under different baselines when customizing DINOv2 and Qwen2.5.

potential as a practical framework for decentralized customization of diverse pretrained models in Web3 environments.

5.6 Ablation Study

This section conducts five ablation experiments to demonstrate the contribution of DPL and MSV schemes for effective lightweight training and verification and the necessity of using TCC with EDI for efficient consensus. All experiments use FedISIC (vision), n2c2 (NER), and DDI (RE) as representative tasks.

5.6.1 Effect of DPL's Slice Selection. We first compare DPL's slice selection method against the following alternative methods for determining which H-layers to train in each round: (i) three static methods, namely *T-Only* (training only the top layers), *B-Only* (training only the bottom layers), and *OT* (training both top and bottom layers simultaneously) [112]; (ii) four dynamic methods, namely *DyTB* (progressively training from top to bottom layers), *DyBT* (from bottom to top layers), and *DyOT* (from both ends toward the middle layers) [112]; and (iii) *Random* (randomly selecting a slice to train in each round).

Table 7 reports the final model performance under different slice selection methods. DPL's method consistently outperforms all alternatives across the three tasks. In contrast, the alternatives exhibit unstable performance across tasks. The static methods (T-Only, B-Only, OT) update only a fixed subset of layers throughout the entire model customization process, failing to explore other potentially informative parts of the model. The dynamic methods (DyTB, DyBT, DyOT) progressively cover more layers over time, yet they follow fixed heuristic traversal orders that do not adapt to the actual data distribution and training progress of model customization tasks. The random method, while able to explore diverse slices across rounds, lacks any principled guidance to prioritize slices with higher utility. DPL's superiority stems from its principled importance scoring, which jointly considers both performance improvement and parameter divergence, enabling the requester to prioritize training slices that yield the highest customization utility.

Table 7: Effect of DPL's importance-score-based slice selection on model performance.

Methods	FedISIC (ACC)	n2c2 (F1)	DDI (F1)
T-Only	73.8%	97.2%	81.6%
B-Only	95.9%	96.1%	86.9%
OT	87.1%	96.9%	76.2%
DyTB	91.7%	97.2%	93.1%
DyBT	96.0%	96.1%	86.9%
DyOT	91.3%	96.9%	88.5%
Random	89.5%	95.8%	83.9%
DPL	98.3%	97.9%	96.1%

5.6.2 Effect of Emulators. We then study the effect of the emulators in approximating the input-output mappings of the global H-layers excluded from the compact model, thereby ensuring effective lightweight training and verification. We vary L_E, the number of H-layers in an emulator, from 0 (no emulator) to 2.

Table 8 shows the resulting model performance. Without an emulator ($L_E = 0$), performance drops significantly. In this setting, any update produced or verified on the compact model is derived from incomplete information. Integrating such an update into the full global model disrupts the original cross-layer semantic information flow of that model. Increasing L_E from 1 to 2 yields limited benefits or even a slight degradation in the final model performance. This confirms that a single-layer emulator produced by KD suffices to capture the critical mappings of the global H-layers excluded from compact models. Moreover, larger emulators are not preferred for two reasons. First, they reduce the number of trainable/verifiable parameters that can fit into the compact model under the fixed hardware budgets of workers and validators, undermining the resource utilization efficiency. Second, they increase the number of candidate slices that the requester must evaluate during slice selection, as the global model must be partitioned more finely when workers can accommodate fewer trainable layers. The above results confirm the effectiveness of emulators in approximating input-output mappings of the global H-layers excluded from the compact model, and demonstrate that our default choice $L_E = 1$ balances approximation fidelity and participant-side resource efficiency. Exploring more compact and advanced emulator architectures or data-efficient distillation methods that reduce the requester's overhead represents a promising direction for future work.

5.6.3 Effect of MSV's Compact-model-based Verification. Next, we explore the effectiveness of using a compact validation model for verification in MSV by comparing it against full-model verification

Table 8: Effect of emulators (consisting of L_E H-layers) on model performance.

L_E	FedISIC (ACC)	n2c2 (F1)	DDI (F1)
0	55.3%	67.1%	63.8%
1 (Default)	98.3%	97.9%	96.1%
2	98.4%	97.1%	96.3%

Table 9: Effect of MSV's compact-model verification on model performance and security against poisoning attacks from malicious workers.

Methods	ACC/F1			*RMW*		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
Full-Verify	98.5%	98.5%	96.6%	0.0%	0.0%	0.0%
MSV	98.3%	97.9%	96.1%	0.0%	0.0%	0.0%

(Full-Verify), in which validators evaluate each worker's update using the full global model. Table 9 reports the performance of the final model and the fraction of the rewards claimed by malicious workers. MSV's results are comparable to those of Full-Verify across all three tasks, indicating the effectiveness of using a compact validation model as a substitute for the full global model for verification. These results empirically justify MSV's design: inserting an emulator into the compact model captures sufficient information about the global H-layers excluded from that model and thus enables effective verification on that model, and using a dynamic threshold compensates for residual bias. Together, MSV enables hardware-constrained validators to participate without sacrificing security or model quality.

5.6.4 Effect of MSV's Dynamic Validation Threshold. We further study the contribution of MSV's dynamic validation threshold by comparing it against a fixed-threshold variant. In this variant, the model requester sets ϵ_j to a constant value at the first round and keeps it unchanged throughout the entire customization process.

Table 10 reports the performance of the final model and the fraction of rewards claimed by malicious workers. With a fixed threshold, the model performance drops below 96% of the ideal ACC/F1, and malicious workers claim about 10% of rewards. The fixed threshold performs poorly because, as the global model improves over rounds, the fixed threshold becomes increasingly lax, making it easier for malicious workers to produce poisoned updates whose validation performance exceeds the threshold. In contrast, the dynamic threshold maintains ACC/F1 above 96% of the ideal and full reward protection across tasks. These results demonstrate that a dynamic validation threshold is crucial for maintaining both model utility and system security, as it tracks the evolving performance of the global model and closes the window that attackers would otherwise exploit.

5.6.5 Necessity of integrating TCC with EDI.. Finally, we examine the role of EDI, particularly its penalty design, in preventing conflict generation and thereby ensuring the efficiency of TCC. We compare DICE (TCC+EDI) against a variant without EDI's penalty (TCC-only) under environments with and without malicious validators. To highlight the effect of EDI, we configure all malicious validators in this experiment to trigger conflicts via poisoning (i.e., submitting

Table 10: Effect of dynamic validation threshold on model performance and security against poisoning attacks from malicious workers.

Methods	ACC/F1			*RMW*		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
Fixed	92.7%	95.4%	92.5%	11.2%	8.7%	9.6%
Dynamic	98.3%	97.9%	96.1%	0.0%	0.0%	0.0%

Table 11: Effect of EDI on ensuring TCC's efficiency. The reported results are the average proportion of sub-tasks executed by each honest validator out of all VA sub-tasks in each round.

Methods	All Honest			20% Malicious		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
TCC-only	6.3%	10.7%	10.5%	90.5%	88.9%	89.9%
TCC+EDI	6.1%	10.5%	10.6%	6.1%	10.6%	10.5%

VA results that deviate from the consensus) whenever profitable, as fraud attacks (submitting copied results) do not generate conflicts.

Table 11 reports the average proportion of VA sub-tasks executed by each honest validator out of all sub-tasks in each round. When all validators are honest, TCC-only and TCC+EDI perform similarly, as no sub-task becomes controversial in this case and thus no extra mandatory workload is incurred for each honest validator. However, when 20% of the validators are malicious, the performance of the two variants diverges sharply. Under TCC-only, the average proportion of VA sub-tasks executed by each honest validator increases by more than 75% across the three tasks. This is because malicious validators deliberately submit conflicting results to obtain extra time in completing more sub-tasks for more rewards, and thus increase the number of conflict-related sub-tasks that require committee voting. In contrast, TCC+EDI performs similarly under both no-adversarial and adversarial environments, with the slight difference attributable to experimental variance. With EDI, the penalty for submitting invalid results exceeds the maximum potential reward obtainable from triggering conflicts. Consequently, malicious validators have no incentive to provoke controversies, and no additional sub-tasks require committee voting beyond the root sub-tasks. The above results validate that EDI is a necessary complement to TCC, ensuring that the efficiency gains of selective committee voting are preserved in adversarial environments.

6 Related Work

The work related to DICE mainly falls into three categories: classical federated learning, blockchain-powered federated learning, and efficient customization techniques.

Classical federated learning aims to train a shared global model from data distributed across different workers while protecting data privacy. The first federated learning algorithm, FedAvg [67], and many of its follow-up works [45, 52, 106] proposed an iterative learning scheme that employs a central server to coordinate workers. Data privacy protection is achieved by transmitting only the model parameters, not raw data, between the server and workers. Classical federated learning algorithms are designed for scenarios where at least one participant, e.g., the server, is trusted.

In contrast, our work is designed for Web3 environments where any participant is untrusted.

Blockchain-powered federated learning aims to enable untrusted Web3 participants to contribute to training models through blockchain techniques [54, 64, 82]. Considering the untrusted yet rational and self-interested characteristics of Web3 participants, many studies have proposed (i) well-designed malicious behavior handling schemes [56, 130] that rely on a cross-node consensus algorithm to produce correct global models, and (ii) incentive schemes [42, 56, 125, 130] that incentivize self-interested entities to participate and behave honestly by providing monetary rewards (e.g., cryptocurrency) proportional to their contributions, rather than relying solely on non-monetary incentives such as reputation or service access [42]. The difference between existing BCFL studies and ours is that these prior studies are designed for small models with at most tens of millions of parameters, such as CNN [56, 66], LSTM [56, 66], ResNet [14, 66], while DICE is designed for pretrained PMs having at least 100 million parameters, e.g., BERT [25], ViT [110, 111], Dinov [78], Qwen [97, 115], LLaMa [100, 129], CLIP [83]. DICE lowers the participant barrier and breaks the scalability bottleneck of the current BCFL paradigm, enabling the system to scale to more workers for better model performance and to more validators for more robust security.

Efficient customization techniques aim to reduce the computational and memory overhead of further pretraining or fine-tuning PMs. These techniques typically follow two paradigms: (i) parameter-efficient fine-tuning [131], such as LoRA [40], Adapter [39], Prompt Tuning [49], and BitFit [11], which preserve the original model structure and parameter bit-width; (ii) model compression [114], such as pruning [41, 61, 112], knowledge distillation [37, 89, 111], and quantization [35, 55], which alter the model structure or reduce parameter bit-width. Many studies have integrated these techniques into federated learning to enable hardware-constrained participants to contribute [27]. However, most of them focus on centralized settings, where a powerful trusted server is adopted to verify or aggregate all worker updates [103, 107, 113]. In contrast, DICE is designed for decentralized BCFL systems [24, 54, 56, 59, 84, 108], where verification and aggregation are performed by validators, each of whom is untrusted, potentially hardware-constrained, and required to redundantly execute the same VA tasks to ensure the validity of cross-validator consensus. For existing BCFL systems, the per-validator workload that grows linearly with worker scale becomes the dominant bottleneck hindering them from scaling to a larger number of data holders. Moreover, this cannot be simply resolved by integrating efficient model customization methods.

7 Conclusion

This paper presented DICE, a novel BCFL algorithm designed for efficient and scalable customization of pretrained models in Web3. We systematically analyzed the efficiency and scalability challenges that prevent existing BCFL systems from being directly applied to customize large PMs. To overcome these limitations, we introduced two sets of novel techniques, namely DPL with MSV to support lightweight training and verification of large PMs, and TCC with EDI to resolve the scalability bottleneck that prior BCFL systems cannot address even when enhanced by existing efficient customization methods. We presented a rigorous theoretical analysis of DICE's efficiency, scalability, and security. Extensive experiments on customizing vision and language PMs corroborated our analysis, showing that compared to the current BCFL paradigm, DICE significantly reduces costs for both workers and validators while maintaining competitive model performance and robust security. Moreover, DICE is compatible with existing efficient customization solutions, accommodating diverse device constraints and model performance requirements without compromising security.

Acknowledgments

This work was funded by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM103) and the Leading Talent of Technological Innovation Program (No. 2023R5214) of Zhejiang Province. We also acknowledge anonymous reviewers for their valuable feedback.

References

[1] 2022. FedISIC2019. https://owkin.github.io/FLamby/fed_isic.html
[2] Aftab Akram, Clémentine Gritti, Mohd Hazali Mohamed Halip, Nur Diyana Kamarudin, Marini Mansor, Syarifah Bahiyah Rahayu, and Melek Önen. 2025. Robust blockchain-based federated learning. In *ICISSP 2025, 11th International Conference on Information Systems Security and Privacy, 20-22 February 2025, Porto, Portugal*, Springer (Ed.).
[3] Kunbolat Algazy, Kairat Sakan, Nursulu Kapalova, Saule Nyssanbayeva, and Dilmukhanbet Dyusenbayev. 2022. Differential Analysis of a Cryptographic Hashing Algorithm HBC-256. *Applied Sciences* 12, 19 (2022).
[4] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. 2013. RBFT: Redundant byzantine fault tolerance. In *2013 IEEE 33rd international conference on distributed computing systems*. IEEE, 297–306.
[5] PureFed Authors. 2024. PureFed: An Efficient Collaborative and Trustworthy Federated Learning Framework Based on Blockchain Network. *IEEE Access* 12 (2024), 82413–82426.
[6] TBCIM Authors. 2025. TBCIM: Two-Level Blockchain-Aided Edge Resource Allocation Mechanism for Federated Learning Service Market. *IEEE Transactions on Networking* (2025).
[7] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How To Backdoor Federated Learning. In *AISTATS*, Vol. 108. 2938–2948.
[8] Yu bao Yan, Chun-Mei Feng, Yuexiang Li, Jinheng Xie, Jun Chen, Mohamed Elhoseiny, Ming Hu, Kaishun Wu, and Lei Zhu. 2025. Temporal Model-Based Federated Active Medical Image Classification. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*. https://api.semanticscholar.org/CorpusID:281830599
[9] Marc Bara. 2025. PromptChain: A Decentralized Web3 Architecture for Managing AI Prompts as Digital Assets. *arXiv preprint arXiv:2507.09579* (2025).
[10] Ahmed Ayoub Bellachia, Mouhamed Amine Bouchiha, Yacine Ghamri-Doudane, and Mourad Rabah. 2025. VerifBFL: Leveraging zk-SNARKs for a Verifiable Blockchained Federated Learning. In *NOMS 2025-2025 IEEE Network Operations and Management Symposium*. 01–09.
[11] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 1–9.
[12] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. 2019. Analyzing Federated Learning through an Adversarial Lens. In *ICML*, Vol. 97. 634–643.
[13] Avik Bhattacharjee. 2025. Overview of Model Customization. In *A Practical Guide to Generative AI Using Amazon Bedrock: Building, Deploying, and Securing Generative AI Applications*. Springer, 249–281.
[14] Genqing Bian, Wenjing Qu, and Bilin Shao. 2023. Blockchain-based trusted federated learning with pre-trained models for COVID-19 detection. *Electronics* 12, 9 (2023), 2068.
[15] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon,

Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Koulako Bala Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel J. Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. *ArXiv* abs/2108.07258 (2021). https://api.semanticscholar.org/CorpusID:237091588

[16] Vitalik Buterin. 2014. A next-generation smart contract and decentralized application platform.

[17] Emanuele Carlini, Patrizio Dazzi, Luca Ferrucci, Jacopo Massa, and Matteo Mordacchini. 2025. Dynamic workload balancing in decentralized edge systems: A marginal cost approach. *Future Generation Computer Systems* (2025), 108167.

[18] Jane Carney, Kushal Upreti, Gaby G Dagher, and Tim Andersen. 2025. FIDELIS: Blockchain-Enabled Protection Against Poisoning Attacks in Federated Learning. *arXiv preprint arXiv:2508.10042* (2025).

[19] Sarthak Chakraborty and Sandip Chakraborty. 2022. Proof of federated training: accountable cross-network model training and inference. In *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–9.

[20] Yuxia Chang, Chen Fang, and Wenzhuo Sun. 2021. A Blockchain-Based Federated Learning Method for Smart Healthcare. *Computational Intelligence and Neuroscience* 2021, 1 (2021), 4376418.

[21] Guorong Chen, Chao Li, Wei Wang, Li Duan, Bin Wang, Zhen Han, and Xiangliang Zhang. 2025. FairReward: Towards Fair Reward Distribution Using Equity Theory in Blockchain-Based Federated Learning. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2025), 1612–1626.

[22] Leiming Chen, Dehai Zhao, Liping Tao, Kai Wang, Sibo Qiao, Xingjie Zeng, and Chee Wei Tan. 2025. A Credible and Fair Federated Learning Framework Based on Blockchain. *IEEE Transactions on Artificial Intelligence* 6, 2 (2025), 301–316.

[23] Jiao Dai, He Huang, Yufei Wang, Li Duan, Fuqiang Hu, Dantong Yan, Zhichen Liu, and Wei Wang. 2025. Byzantine-resistant model verification for decentralized federated learning in intelligent transportation systems. *Applied Soft Computing* 169 (2025), 111852.

[24] Harsh Bimal Desai, Mustafa Safa Ozdayi, and Murat Kantarcioglu. 2021. Blockfla: Accountable federated learning via hybrid blockchain architecture. In *Proceedings of the eleventh ACM conference on data and application security and privacy*. 101–112.

[25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805

[26] John R. Douceur. 2002. The Sybil Attack. In *Peer-to-Peer Systems*. Springer Berlin Heidelberg, 251–260.

[27] Tao Fan, Hanlin Gu, Xuemei Cao, Chee Seng Chan, Qian Chen, Yiqiang Chen, Yihui Feng, Yang Gu, Jiaxiang Geng, Bing Luo, et al. 2025. Ten challenging problems in federated foundation models. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[28] Lei Feng, Yiqi Zhao, Shaoyong Guo, Xuesong Qiu, Wenjing Li, and Peng Yu. 2022. BAFL: A Blockchain-Based Asynchronous Federated Learning Framework . *IEEE Trans. Comput.* 71, 05 (2022), 1092–1103.

[29] Ocean Protocol Foundation. 2020. Ocean Protocol: A Decentralized Data Exchange Protocol to Unlock Data for AI. https://oceanprotocol.com/tech-whitepaper.pdf

[30] Wenyu Gao, Gaochao Xu, and Xianqiu Meng. 2025. FedCon: Scalable and Efficient Federated Learning via Contribution-Based Aggregation. *Electronics* 14, 5 (2025).

[31] Stefanos Georgiou, Maria Kechagia, and Federica Sarro. 2022. Green AI: Do Deep Learning Frameworks Have Different Costs? *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (2022), 1082–1094. https://api.semanticscholar.org/CorpusID:247614182

[32] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. *ACM Trans. Comput. Healthcare* 3, 1 (2021).

[33] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. 2024. Decentralized federated learning: A survey on security and privacy. *IEEE Transactions on Big Data* 10, 2 (2024), 194–213.

[34] Heiko Hamann and Andreagiovanni Reina. 2021. Scalability in computing and robotics. *IEEE Trans. Comput.* 71, 6 (2021), 1453–1465.

[35] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).

[36] María Herrero-Zazo, Isabel Segura-Bedmar, Paloma Martínez, and Thierry Declerck. 2013. The DDI corpus: An annotated corpus with pharmacological substances and drug–drug interactions. *Journal of biomedical informatics* 46, 5 (2013), 914–920.

[37] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).

[38] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).

[39] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*.

[40] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. LoRA: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[41] Lanxiang Hu, Tajana Rosing, and Hao Zhang. 2025. TrimLLM: Progressive Layer Dropping for Domain-Specific LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 667–681.

[42] Cornelius Ihle, Dennis Trautwein, Moritz Schubotz, Norman Meuschke, and Bela Gipp. 2023. Incentive Mechanisms in Peer-to-Peer Networks — A Systematic Literature Review. *Comput. Surveys* 55, 14s (2023).

[43] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. Tinybert: Distilling bert for natural language understanding. In *Findings of the association for computational linguistics: EMNLP 2020*. 4163–4174.

[44] Aditya Pribadi Kalapaaking, Ibrahim Khalil, and Xun Yi. 2023. Blockchain-based federated learning with SMPC model verification against poisoning attack for healthcare systems. *IEEE Transactions on Emerging Topics in Computing* 12, 1 (2023), 269–280.

[45] Sai Praneeth Karimireddy, S. Kale, M. Mohri, Sashank J. Reddi, Sebastian U. Stich, and A. T. Suresh. 2020. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *ICML*.

[46] Jin-Dong Kim, Tomoko Ohta, Yoshimasa Tsuruoka, Yuka Tateisi, and Nigel Collier. 2004. Introduction to the Bio-Entity Recognition Task at JNLPBA. In *Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and its Applications (NLPBA/BioNLP)*. COLING, Geneva, Switzerland, 73–78.

[47] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.

[48] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* 36, 4 (2020), 1234–1240.

[49] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 3045–3059.

[50] Dun Li, Dezhi Han, Tien-Hsiung Weng, Zibin Zheng, Hongzhi Li, Han Liu, Arcangelo Castiglione, and Kuan-Ching Li. 2022. Blockchain for federated learning toward secure distributed machine learning systems: a systemic survey. *Soft Computing* 26, 9 (2022), 4423–4440.

[51] Kai Li, Zhicai Zhang, and Fang Fu. 2023. An Incentive Mechanism for Consortium Blockchain-based Cross-Silo Federated Learning. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. 451–458.

[52] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated Optimization in Heterogeneous Networks. In *MLSys*, Vol. 2. 429–450.

[53] Wenyu Li, Chenglin Feng, Lei Zhang, Hao Xu, Bin Cao, and Muhammad Ali Imran. 2020. A scalable multi-layer PBFT consensus for blockchain. *IEEE Transactions on Parallel and Distributed Systems* 32, 5 (2020), 1146–1160.

[54] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. 2021. A Blockchain-Based Decentralized Federated Learning Framework with Committee Consensus. *IEEE Network* 35, 1 (2021), 234–241.

[55] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* 461 (2021), 370–403.

[56] Hong Lin, Ke Chen, Dawei Jiang, Lidan Shou, and Gang Chen. 2024. Refiner: a reliable and efficient incentive-driven federated learning system powered by blockchain. *The VLDB Journal* 33, 3 (2024), 807–831.

[57] Hong Lin, Shixin Wan, Zhongle Xie, Ke Chen, Meihui Zhang, Lidan Shou, and Gang Chen. 2025. A Comprehensive Study of Shapley Value in Data Analytics. *Proc. VLDB Endow.* 18, 9 (2025), 3077–3092.

[58] Hong Lin, Zhongle Xie, Lidan Shou, Ke Chen, and Gang Chen. 2026. DICE: Decentralized Incentive-driven Federated Learning for Efficient & Scalable Customization of Pretrained Models in Web3. https://github.com/Hoho-Lin/DICE

[59] Yuan Liu, Zhengpeng Ai, Shuai Sun, Shuangfeng Zhang, Zelei Liu, and Han Yu. 2020. *FedCoin: A Peer-to-Peer Payment System for Federated Learning.* 125–138.

[60] Charles Lu and Jayasheree Kalpathy-Cramer. 2022. Distribution-Free Federated Learning with Conformal Predictions. arXiv:2110.07661

[61] Yao Lu, Hao Cheng, Yujie Fang, Zeyu Wang, Jiaheng Wei, Dongwei Xu, Qi Xuan, Xiaoniu Yang, and Zhaowei Zhu. 2024. Reassessing layer pruning in LLMs: New insights and methods. *arXiv preprint arXiv:2411.15558* (2024).

[62] Yuan Lu, Qiang Tang, and Guiling Wang. 2018. On enabling machine learning tasks atop public blockchains: A crowdsourcing approach. In *2018 IEEE international conference on data mining workshops (ICDMW)*. IEEE, 81–88.

[63] Elizabeth Lui and Jiahao Sun. 2025. Bittensor Protocol: The Bitcoin in Decentralized Artificial Intelligence? A Critical and Empirical Analysis. *arXiv preprint arXiv:2507.02951* (2025).

[64] Chuan Ma, Jun Li, Long Shi, Ming Ding, Taotao Wang, Zhu Han, and H. Vincent Poor. 2022. When Federated Learning Meets Blockchain: A New Distributed Learning Paradigm. *IEEE Computational Intelligence Magazine* 17, 3 (2022), 26–33.

[65] Xuyang Ma, Du Xu, and Katinka Wolter. 2024. Towards blockchain-enabled decentralized and secure federated learning. *Information Sciences* 662 (2024).

[66] T Manoj, Krishnamoorthi Makkithaya, and VG Narendra. 2025. A blockchain-assisted trusted federated learning for smart agriculture. *SN Computer Science* 6, 3 (2025), 221.

[67] H. B. McMahan, Eider Moore, Daniel Ramage, S.C.D. Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*, Vol. 54. 1273–1282.

[68] MedSynapse. 2026. MedSynapse Official Website. https://ethglobal.com/showcase/medsynapse-kyw2m

[69] Brandon Alejandro Mosqueda González, Omar Hasan, and Lionel Brunie. 2025. Mitigation of Sybil-based Poisoning Attacks in Permissionless Decentralized Learning. In *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–5. https://hal.science/hal-05002718

[70] Noorain Mukhtiar, Adnan Mahmood, Yipeng Zhou, Jian Yang, Jing Teng, and Quan Z. Sheng. 2024. Federated Learning at the Forefront of Fairness: A Multifaceted Perspective. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:281484496

[71] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System.

[72] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.

[73] Keerthi Nelaturu, Sidi Mohamed Beillahi, Fan Long, and Andreas G. Veneris. 2021. Smart Contracts Refinement for Gas Optimization. *BRAINS* (2021), 229–236.

[74] Mind Network. 2026. Product of Mind Network: World AI Health Hub. https://docs.mindnetwork.xyz/minddocs/product/agenticworld/agenticworld-user-guide/world-ai-health-hub

[75] Dinh C Nguyen, Ming Ding, Quoc-Viet Pham, Pubudu N Pathirana, Long Bao Le, Aruna Seneviratne, Jun Li, Dusit Niyato, and H Vincent Poor. 2021. Federated learning meets blockchain in edge computing: Opportunities and challenges. *IEEE Internet of Things Journal* 8, 16 (2021), 12806–12825.

[76] Quang-Thang Nguyen, Bao Son Do, Thi Tam Nguyen, and Ba-Lam Do. 2022. GasSaver: A Tool for Solidity Smart Contract Optimization. *ACM BSCI* (2022).

[77] Weiguang Ning, Yingjuan Zhu, Caixia Song, Hongxia Li, Lihui Zhu, Jinbao Xie, Tianyu Chen, Tong Xu, Xi Xu, and Jiwei Gao. 2024. Blockchain-Based Federated Learning: A Survey and New Perspectives. *Applied Sciences* 14, 20 (2024).

[78] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2023. DINOv2: Learning Robust Visual Features without Supervision.

[79] Le Peng, Gaoxiang Luo, Sicheng Zhou, Jiandong Chen, Ziyue Xu, Ju Sun, and Rui Zhang. 2024. An in-depth evaluation of federated learning on biomedical natural language processing for information extraction. *npj Digit. Medicine* 7, 1 (2024).

[80] Le Peng, sicheng zhou, jiandong chen, Rui Zhang, Ziyue Xu, and Ju Sun. 2023. A Systematic Evaluation of Federated Learning on Biomedical Natural Language Processing. https://openreview.net/forum?id=pLEQFXACNA

[81] Sara Pieri, Jose Renato Restom, Samuel Horvath, and Hisham Cholakkal. 2023. Handling data heterogeneity via architectural design for federated visual recognition. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*.

[82] Youyang Qu, Md Palash Uddin, Chenquan Gan, Yong Xiang, Longxiang Gao, and John Yearwood. 2022. Blockchain-enabled Federated Learning: A Survey. *ACM Comput. Surv.* 55, 4 (2022), 35 pages.

[83] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:231591445

[84] Paritosh Ramanan and Kiyoshi Nakayama. 2020. BAFFLE : Blockchain Based Aggregator Free Federated Learning. In *2020 IEEE International Conference on Blockchain (Blockchain)*. 72–81.

[85] Shuyang Ren, Eunsam Kim, and Choonhwa Lee. 2024. A scalable blockchain-enabled federated learning architecture for edge computing. *Plos one* 19, 8 (2024), e0308991.

[86] Yanli Ren, Mingqi Hu, Zhe Yang, Guorui Feng, and Xinpeng Zhang. 2024. BPFL: Blockchain-based privacy-preserving federated learning against poisoning attack. *Information Sciences* 665 (2024), 120377.

[87] Phillip Rogaway and Thomas Shrimpton. 2004. Cryptographic Hash-Function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-Preimage Resistance, and Collision Resistance. In *Fast Software Encryption*. Springer Berlin Heidelberg, Berlin, Heidelberg, 371–388.

[88] Fahad Saleh. 2021. Blockchain without waste: Proof-of-stake. *The Review of Financial Studies* 34, 3 (2021), 1156–1190.

[89] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[90] Johannes Schneider, Christian Meske, and Pauline Kuss. 2024. Foundation models: A new paradigm for artificial intelligence. *Business & Information Systems Engineering* 66, 2 (2024), 221–231.

[91] Neha Sharma, Vibhor Jain, and Anju Mishra. 2018. An Analysis Of Convolutional Neural Networks For Image Classification. *Procedia Computer Science* 132 (2018), 377–384. International Conference on Computational Intelligence and Data Science.

[92] Muhammad Shayan, Clement Fung, Chris JM Yoon, and Ivan Beschastnikh. 2020. Biscotti: A blockchain system for private and secure federated learning. *IEEE Transactions on Parallel and Distributed Systems* 32, 7 (2020), 1513–1525.

[93] Meng Shen, Zhehui Tan, Dusit Niyato, Yuzhi Liu, Jiawen Kang, Zehui Xiong, Liehuang Zhu, Wei Wang, and Xuemin Shen. 2024. Artificial Intelligence for Web 3.0: A Comprehensive Survey. *Comput. Surveys* 56, 10 (2024), 1–39.

[94] Zhanming Shen, Tianqi Xu, Hao Wang, Jian Li, and Miao Pan. 2025. pFedGPT: Hierarchically Optimizing L,oRA, Aggregation Weights for Personalized Federated GPT, Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 4766–4778.

[95] Olivier Taboureau, Sonny Kim Nielsen, Karine Audouze, Nils Weinhold, Daniel Edsgä, rd, Francisco S Roque, Irene Kouskoumvekaki, Alina Bora, Ramona Curpan, t Jensen, Thomas Skø, et al. 2010. ChemProt: a disease chemical biology database. *Nucleic acids research* 39, suppl_1 (2010), D367–D372.

[96] Nurbek Tastan, Samar Fares, Toluwani Aremu, Samuel Horvath, and Karthik Nandakumar. 2024. Redefining contributions: shapley-driven federated learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. Article 554, 9 pages.

[97] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/

[98] Youliang Tian, Ta Li, Jinbo Xiong, Md Zakirul Alam Bhuiyan, Jianfeng Ma, and Changgen Peng. 2021. A blockchain-based machine learning framework for edge services in IIoT. *IEEE Transactions on Industrial Informatics* 18, 3 (2021), 1918–1929.

[99] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*. PMLR, 10347–10357.

[100] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv* (2023). arXiv:2302.13971

[101] Kentaroh Toyoda, Jun Zhao, Allan Neng Sheng Zhang, and P Takis Mathiopoulos. 2020. Blockchain-enabled federated learning with mechanism design. *IEEE Access* 8 (2020), 219744–219756.

[102] Nguyen Truong, Kai Sun, Siyao Wang, Florian Guitton, and YiKe Guo. 2021. Privacy preservation in federated learning: An insightful survey from the GDPR

perspective. *Computers & Security* 110 (2021), 102402.

[103] Adam Tupper and Christian Gagn'e. 2025. Personalized Federated Fine-Tuning of Vision Foundation Models for Healthcare. *ArXiv* abs/2510.12741 (2025). https://api.semanticscholar.org/CorpusID:282064942

[104] Muhammad Habib ur Rehman, Ahmed Mukhtar Dirir, Khaled Salah, Ernesto Damiani, and Davor Svetinovic. 2021. TrustFed: A framework for fair and trustworthy cross-device federated learning in IIoT. *IEEE Transactions on Industrial Informatics* 17, 12 (2021), 8485–8494.

[105] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser, Ł, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[106] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated Learning with Matched Averaging. In *ICLR*.

[107] Xin'ao Wang, Huan Li, Ke Chen, and Lidan Shou. 2023. FedBFPT: An Efficient Federated Learning Framework for BERT Further Pre-training. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:260859911

[108] Jiasi Weng, Jian Weng, Jilian Zhang, Ming Li, Yue Zhang, and Weiqi Luo. 2021. DeepChain: Auditable and Privacy-Preserving Deep Learning with Blockchain-Based Incentive. *IEEE Trans. Dependable Secur. Comput.* 18, 5 (2021), 2438–2455.

[109] Francesc Wilhelmi, Lorenza Giupponi, and Paolo Dini. 2021. Blockchain-enabled server-less federated learning. *arXiv preprint arXiv:2112.07938* (2021), 1–14.

[110] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. 2020. Visual Transformers: Token-based Image Representation and Processing for Computer Vision. arXiv:2006.03677 [cs.CV]

[111] Kan Wu, Jinnian Zhang, Houwen Peng, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. 2022. Tinyvit: Fast pretraining distillation for small vision transformers. In *European conference on computer vision*. Springer, 68–85.

[112] Guangxuan Xiao, Ji Lin, and Song Han. 2023. Offsite-tuning: Transfer learning without full model. *arXiv preprint arXiv:2302.04870* (2023).

[113] Honghui Xu, Shiva Shrestha, Wei Chen, Zhiyuan Li, and Zhipeng Cai. 2025. DP-FedLoRA: Privacy-Enhanced Federated Fine-Tuning for On-Device Large Language Models. *arXiv preprint arXiv:2509.09097* (2025).

[114] Mengwei Xu, Dongqi Cai, Wangsong Yin, Shangguang Wang, Xin Jin, and Xuanzhe Liu. 2025. Resource-efficient Algorithms and Systems of Foundation Models: A Survey. *ACM Comput. Surv.* 57, 5, Article 110 (Jan. 2025), 39 pages. https://doi_.org/10.1145/3706418

[115] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[116] Bang Yang, Asif Raza, Yuexian Zou, and Tong Zhang. 2023. Customizing general-purpose foundation models for medical report generation. *arXiv preprint arXiv:2306.05642* (2023).

[117] Jiancheng Yang, Rui Shi, and Bingbing Ni. 2021. MedMNIST Classification Decathlon: A Lightweight AutoML Benchmark for Medical Image Analysis. In *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*. 191–195.

[118] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. 2023. MedMNIST v2-A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data* 10, 1 (2023), 41.

[119] Qing Yang, Wei Xu, Taotao Wang, Hao Wang, Xiaoxiao Wu, Bin Cao, and Shengli Zhang. 2024. Blockchain-Based Decentralized Federated Learning With On-Chain Model Aggregation and Incentive Mechanism for Industrial IoT. *IEEE Open Journal of the Communications Society* 5 (2024), 6420–6429.

[120] Qing Yang, Wei Xu, Taotao Wang, Hao Wang, Xiaoxiao Wu, Bin Cao, and Shengli Zhang. 2024. Blockchain-Based Decentralized Federated Learning With On-Chain Model Aggregation and Incentive Mechanism for Industrial IoT. *IEEE Open Journal of the Communications Society* 5 (2024), 6420–6429.

[121] Xiaohui Yang and Tianchang Li. 2024. A Blockchain-based federated learning framework for secure aggregation and fair incentives. *Connection Science* 36 (2024). https://api.semanticscholar.org/CorpusID:267970430

[122] Mang Ye, Wenke Huang, Zekun Shi, He Li, and Bo Du. 2025. Revisiting federated learning with label skew: an over-confidence perspective. *Science China Information Sciences* 68 (2025). https://api.semanticscholar.org/CorpusID:279980636

[123] Musa Yenilmez and Ilhan Aydin. 2025. Particle Swarm Optimized Federated Learning for Efficient Classification of Medical Images. In *2025 15th International Conference on Advanced Computer Information Technologies (ACIT)*. 868–872.

[124] Guangsheng Yu, Xu Wang, Caijun Sun, Qin Wang, Ping Yu, Wei Ni, and Ren Ping Liu. 2023. IronForge: An open, secure, fair, decentralized federated learning. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2023), 354–368.

[125] Yufeng Zhan, Jie Zhang, Zicong Hong, Leijie Wu, Peng Li, and Song Guo. 2022. A Survey of Incentive Mechanism Design for Federated Learning. *IEEE Transactions on Emerging Topics in Computing* 10, 2 (2022), 1035–1044.

[126] Bingxue Zhang, Guangguang Lu, Yuncheng Wu, Kunpeng Ren, and Feida Zhu. 2025. Zkfhed: A Verifiable and Scalable Blockchain-Enhanced Federated Learning System. *IEEE Transactions on Knowledge and Data Engineering* 37, 6 (2025), 3841–3854.

[127] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, Qin Xie, Guiming Xie, and Xuejian Gong. 2025. HMI: hierarchical knowledge management for efficient multi-tenant inference in pretrained language models. *The VLDB Journal* 34, 4 (2025), 43.

[128] Peng Zhang, Gang Liu, Zhiyu Chen, Jianwei Guo, and Peng Liu. 2022. A study of a federated learning framework based on the interstellar file system and blockchain: Private Blockchain Federated Learning. In *2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)*. IEEE, 267–273.

[129] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385 [cs.CL]

[130] Zhebin Zhang, Dajie Dong, Yuhang Ma, Yilong Ying, Dawei Jiang, Ke Chen, Lidan Shou, and Gang Chen. 2021. Refiner: a reliable incentive-driven federated learning system powered by blockchain. *Proc. VLDB Endow.* 14, 12 (2021), 2659–2662.

[131] Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models. In *Annual Meeting of the Association of Computational Linguistics 2023*. Association for Computational Linguistics (ACL), 9963–9977.

[132] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. 2023. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *Comput. Surveys* 55, 11 (2023), 1–31.

[133] ZIRCONTech. 2025. The Convergence of AI and Web3: Building the Future of Decentralized Intelligence. https://zircon.tech/blog/the-convergence-of-ai-and-web3-building-the-future-of-decentralized-intelligence/